

รถอัจฉริยะเดินตามป้ายจราจร

SMART CAR FOLLOW SIGN

นาย ศุภเมธ จันทร์สกุล

นาย บุญชู เจ็้ง

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต

ภาควิชาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์

มหาวิทยาลัยบูรพา

ปีการศึกษา 2568

ชื่อโครงการ	รถอัจฉริยะเดินตามป้ายจราจร
โดย	นายศุภเมธ จันทรสกุล นายบุญชู เจ็๋ง
อาจารย์ที่ปรึกษา	รองศาสตราจารย์วิรุฬห์ ศรีบริรักษ์
จำนวนหน้า	75 หน้า
ปีการศึกษา	2568

ภาควิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์ มหาวิทยาลัยบูรพา อนุมัติรายงานสรุปโครงงานฉบับนี้ เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมระบบสมองกลฝังตัว

.....อาจารย์ที่ปรึกษา

(รองศาสตราจารย์วิรุฬห์ ศรีบริรักษ์)

.....หัวหน้าภาควิศวกรรมไฟฟ้า

(อาจารย์ ดร. ณัฐพันธ์ ถนอมสัจย์)

บทคัดย่อ

โครงการนี้เป็นการพัฒนาระบบรถเดินตามป้ายจราจรขนาดเล็กบนบอร์ด Raspberry Pi โดยอาศัยเทคโนโลยีอัลกอริทึมตรวจจับวัตถุและใช้กล้องควบคู่กับเทคโนโลยีทำให้ระบบประมวลผลภาพแบบเรียลไทม์จากกล้องในการตรวจจับและจำแนกป้ายจราจร ที่ติดตั้งในสภาพแวดล้อมที่กำหนดได้อย่างแม่นยำและรวดเร็ว โดยจำแนกการเคลื่อนที่ออกเป็น 4 รูปแบบ ได้แก่ เดินหน้า, หยุด, เลี้ยวซ้าย, และเลี้ยวขวา และการเลี้ยวรถใช้ระบบ Differential Drive ที่ทำงานผ่าน GPIO และเทคนิค PWM เพื่อควบคุมความเร็วและทิศทางของมอเตอร์ ในการดำเนินงานมีการเลือกกล้องตรวจจับที่มีค่าความเชื่อมั่นสูงสุดเพื่อลดการผิดพลาด นอกจากนี้ยังได้ลดระยะเวลาการเลี้ยวและระยะเวลาการป้องกันคำสั่งซ้ำ เพื่อให้รถมีการตอบสนองที่รวดเร็วและต่อเนื่อง ทำให้รถเดินตามป้ายจราจรมีความแม่นยำสูงและนำไปใช้งานได้จริง

คำสำคัญ: การเรียนรู้อัลกอริทึมตรวจจับวัตถุ , เทคนิค PWM , เทคนิค Template Matching

กิตติกรรมประกาศ

รายงานสรุปโครงงานฉบับนี้สำเร็จลุล่วงไปได้ด้วยความช่วยเหลืออย่างดีจากรองศาสตราจารย์ วิรุฬห์ ศรี
บริรักษ์ อาจารย์ที่ปรึกษาโครงงาน ที่ได้ให้คำแนะนำและข้อคิดเห็นต่างๆ ในการทำโครงงานมาโดยตลอด
ขอขอบคุณคณะกรรมการสอบโครงงาน อาจารย์ ดร.ภาณุวัฒน์ ด้านกลางและอาจารย์ ธราธร บุญศรีที่ให้
คำแนะนำที่เป็นประโยชน์ ตลอดจนคณาจารย์ทุกท่านที่ไม่ได้กล่าวนามไว้ ณ ที่นี้ ที่ได้เอื้อเฟื้อสถานที่ทำการ
ทดลอง เครื่องมือและอุปกรณ์ในการดำเนินโครงงาน

สารบัญ

หน้า

บทคัดย่อ	๗
กิตติกรรมประกาศ	๘
สารบัญ	๙
บทที่ 1	9
บทนำ	9
1.1 ที่มาและปัญหา.....	9
1.2 วัตถุประสงค์.....	9
1.3 ขอบเขตการทำโครงการ.....	10
1.4 แผนการดำเนินงาน.....	10
1.5 ประโยชน์ที่คาดว่าจะได้รับ	10
ตารางที่ 1.1 แผนการดำเนินงาน	10
บทที่ 2.....	12
ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	12
2.1 เทคนิคการเขียนโปรแกรมและการควบคุมฮาร์ดแวร์ (Motor Control & Setup)	12
2.1.1 การใช้ Raspberry Pi GPIO และ PWM	12
2.1.2 โมดูลารี่โรเซชันและการแยกส่วนงาน	12
2.1.3 การจัดการทรัพยากรและการปิดระบบอย่างปลอดภัย (Safe Shutdown)	13

2.1.4 การใช้ค่าคงที่และการปรับจูนพารามิเตอร์	14
2.2 เทคนิคการประมวลผลภาพและ YOLO (Vision & AI)	14
2.2.1 การใช้ไลบรารีในการตั้งค่ากล้อง	14
2.2.2 การใช้ YOLO สำหรับการตรวจจับวัตถุ (Object Detection)	14
2.2.3 เทคนิคการจัดการเฟรมและการเพิ่มประสิทธิภาพ (Frame Management & Performance)	15
2.2.4 การจัดการลอจิกการตัดสินใจ (Decision Logic)	15
2.2.5 ลอจิกการแก้ไขผลลัพธ์การตรวจจับ (Special Detection Check)	16
2.2.6 เทคนิคการดีบั๊กและการแสดงผล (Debugging & Visualization)	16
2.3 เทคนิคการทำ Autorun สำหรับสคริปต์ Python บน Raspberry Pi ด้วย systemd	17
2.3.1 การเตรียมสคริปต์ Python	17
2.3.2 การสร้างและกำหนดค่า Service File	17
2.3.3 การเปิดใช้งานและควบคุม Service	17
2.4 เทคนิคหลักการขับเคลื่อน (Differential Drive Kinematics)	18
2.4.1 การควบคุมการเคลื่อนที่พื้นฐาน	18
2.4.2 สมการพื้นฐาน (Kinematic Model)	18
2.4.3 การประยุกต์ใช้กับระบบ YOLO	19
2.5 บอร์ดควบคุม	19
2.5.1 Raspberry Pi	20
2.5.2 Open MV	20
2.5.3 Arduino	20

2.5.4 ทำไมถึงเลือกใช้บอร์ด Raspberry Pi ในการทำรถเดินตามป้ายจราจร	21
2.6 โมดูลขับเคลื่อนมอเตอร์	22
2.6.1 L298N.....	22
2.6.2 TB6612FNG	23
2.6.3 DRV8833	24
2.6.4 ทำไมถึงเลือกใช้โมดูลมอเตอร์ L298N ในการทำรถเดินตามป้าย	25
2.7 การเปรียบเทียบกล้อง Picam และกล้อง Webcam	25
2.7.1 การเชื่อมต่อและการใช้ทรัพยากรฮาร์ดแวร์.....	25
2.7.2 ผลกระทบต่อระบบ YOLOและการตอบสนอง	26
2.7.3 เทคนิคการจัดการภาพ (Frame Management).....	26
2.8 ทำไมถึงเลือกอัลกอริทึมแบบขั้นตอนเดียวในการตรวจจับวัตถุแบบเรียลไทม์.....	27
2.9 อัลกอริทึมแบบขั้นตอนเดียว	27
2.9.1 YOLOv3	27
2.9.2 YOLOv7	27
2.9.3 YOLOv8	27
ตารางการเปรียบเทียบข้อดี ข้อเสียของแต่ละเวอร์ชัน	28
2.9.4 ทำไมถึงเลือกใช้ YOLOv8 ในการทำรถเดินตามป้าย	29
2.10 งานวิจัยและพัฒนาที่คล้ายคลึงกับโครงงานรถเดินตามป้าย	30
2.10.1 โครงงานการตรวจจับป้ายจราจรด้วย YOLOv8 บน Raspberry Pi.....	30
2.10.2 โครงงานหุ่นยนต์ตามมนุษย์โดยใช้ YOLOv5 และสถาปัตยกรรมแบบกระจาย.....	31

บทที่ 3.....	33
หลักการและแนวคิดในการออกแบบระบบ	33
3.1 องค์ประกอบหลักของโครงงาน.....	33
3.2 องค์ประกอบ Vision Layer ของโครงงาน	34
3.3 ภาพรวมการทำงาน	34
3.4 ตารางการต่อขา.....	34
3.5 ตารางวิธีการเคลื่อนที่และเคลื่อนที่ยังไง	35
3.6 สรุปการเคลื่อนที่.....	36
3.7 Pi cam flowchart	37
3.8 สรุปโครงสร้างโค้ด Picam.....	38
3.9 Webcam flowchart	43
3.10 สรุปโครงสร้างของโค้ด Webcam	44
3.11 สรุปโค้ดแยกทดสอบส่วนแสดงผลภาพของกล้องและส่วนในการเชื่อมต่ออัลกอริทึมตรวจจับวัตถุ.....	52
บทที่ 4.....	64
ทดสอบโครงงาน.....	64
4.1 ตาราง Unit test	64
สรุปผลการทดสอบของ unit test.....	64
4.2 สรุปการทดสอบของ systems test	66
4.3 สรุปการทดสอบของโครงงาน.....	66

4.4 รูปการทดสอบ.....	67
บทที่ 5.....	72
บทสรุปและข้อแนะนำ	72
5.1 บทสรุปโครงการ	72
5.2 ข้อเสนอแนะ	73
แหล่งอ้างอิงของโครงการ.....	75

บทที่ 1

บทนำ

1.1 ที่มาและปัญหา

ในปัจจุบัน การพัฒนาเทคโนโลยียานยนต์ขับเคลื่อนอัตโนมัติ (Autonomous Vehicles) เป็นสิ่งจำเป็นอย่างยิ่งต่อระบบขนส่งยุคใหม่ หากระบบเหล่านี้ไม่ได้รับการพัฒนาอย่างต่อเนื่อง อาจส่งผลกระทบต่อความล่าช้าและทำให้เกิดความสูญเสียทางเศรษฐกิจจากการขนส่งที่ไม่มีประสิทธิภาพ เราจึงทำโปรเจกต์จำลองขนาดเล็กนี้ที่ใช้พื้นที่ไม่เยอะมากในการสร้างก่อนนำไปใช้ในขนาดจริง การพัฒนารถยนต์ไร้คนขับขนาดจริงนั้น มีค่าใช้จ่ายสูงมาก และมีความเสี่ยงด้านความปลอดภัยสูงหากเกิดข้อผิดพลาดในการเขียนโปรแกรมควบคุม จากการสืบค้นข้อมูลในทางวิศวกรรมและการศึกษา พบว่าระบบจำลองที่มีอยู่ยังไม่สามารถแก้ไขปัญหาได้อย่างสมบูรณ์ โดยเฉพาะอย่างยิ่งในส่วนของ การจดจำและตอบสนองต่อป้ายจราจร ซึ่งเป็นหัวใจสำคัญของการขับเคลื่อนอัตโนมัติ

เพื่อแก้ไขปัญหานี้ ผู้ออกแบบจึงต้องการ พัฒนารถจำลองขนาดเล็ก ให้มีประสิทธิภาพในการ จดจำและปฏิบัติตามป้ายจราจร ที่สูงขึ้น สามารถตอบสนองต่อคำสั่งที่ซับซ้อนได้อย่างรวดเร็วและแม่นยำ เพื่อใช้เป็นเครื่องมือในการ พัฒนาและทดสอบอัลกอริทึม สำหรับเทคโนโลยียานยนต์ขับเคลื่อนอัตโนมัติอย่างมีประสิทธิภาพมากขึ้นต่อไป

1.2 วัตถุประสงค์

- 1.พัฒนาระบบในการจดจำป้ายจราจร (หยุด ขวา ซ้าย หน้า)
- 2.พัฒนาระบบควบคุมตัวรถ มอเตอร์ต่างๆ
- 3.พัฒนาให้กล้องเห็นป้ายจราจรที่กำหนดเอาไว้ แล้วรถจำลองเคลื่อนไหวตามป้ายที่กำหนด

1.3 ขอบเขตการทำโครงการ

- 1.รถจำลองที่สร้างสามารถเดินตามป้ายจราจร (หยุด ขวา ซ้าย หน้า)ได้
- 2.ทำการเขียนโปรแกรมควบคุมต่างๆในโปรแกรม Python บนบอร์ด Raspberry Pi 5 และบอร์ดขับเคลื่อน (L298N) และนำอุปกรณ์ทั้งหมดมาติดตั้งและประกอบเป็นตัวรถอย่างเสร็จสมบูรณ์

1.4.แผนการดำเนินงาน

แผนการดำเนินงาน (ตาราง1.1) มีขั้นตอนดังนี้

- 1.เข้าพบอาจารย์ที่ปรึกษาเพื่อหาหัวข้อโครงการ
- 2.ศึกษาหลักการทำงานและเทคโนโลยีที่เกี่ยวข้อง
- 3.ออกแบบระบบการทำงานของรถอัจฉริยะ
- 4.พัฒนาโปรแกรมประมวลผลภาพและควบคุมการเคลื่อนที่
- 5.ทดสอบระบบการตรวจจับป้ายและการตอบสนองของรถ
- 6.ปรับปรุงประสิทธิภาพของระบบ
- 7.ทดสอบระบบรวม (Integration Test)
- 8.จัดทำรายงานสรุปผลโครงการ

1.5 ประโยชน์ที่คาดว่าจะได้รับ

- 1.ได้ทักษะและหลักการในการใช้ Raspberry pi และ python
- 2.รู้และเข้าใจหลักการในการทำงานเทคโนโลยี YOLO
- 3.สามารถนำองค์ความรู้ในการทำโปรเจกไปปรับใช้กับการทำงานในอนาคตต่อไปได้

ตารางที่ 1.1 แผนการดำเนินงาน

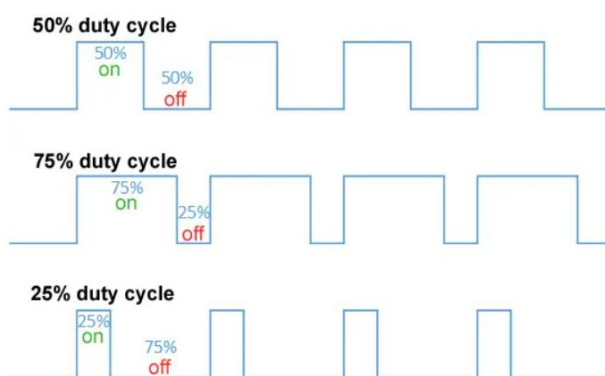
บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

2.1 เทคนิคการเขียนโปรแกรมและการควบคุมฮาร์ดแวร์ (Motor Control & Setup)

2.1.1 การใช้ Raspberry Pi GPIO และ PWM

- การกำหนดพิน (Pin Assignment) : กำหนดพินสำหรับควบคุมทิศทาง (IN1-IN4) และพิน PWM (ENA, ENB) ชัดเจน
- PWM (Pulse Width Modulation) : ใช้คลาส GPIO.PWM เพื่อสร้างสัญญาณ PWM บนพิน ENA (มอเตอร์ A) และ ENB (มอเตอร์ B) เพื่อควบคุม ความเร็ว ของมอเตอร์
- ฟังก์ชันควบคุมมอเตอร์ระดับต่ำ : ฟังก์ชัน `_drive_motor_a` และ `_drive_motor_b` แปลง ทิศทาง (+1, -1, 0) และ ความเร็ว (0-100) ไปเป็นการตั้งค่า GPIO (HIGH/LOW) สำหรับทิศทาง และการเปลี่ยน Duty Cycle ของ PWM สำหรับความเร็ว



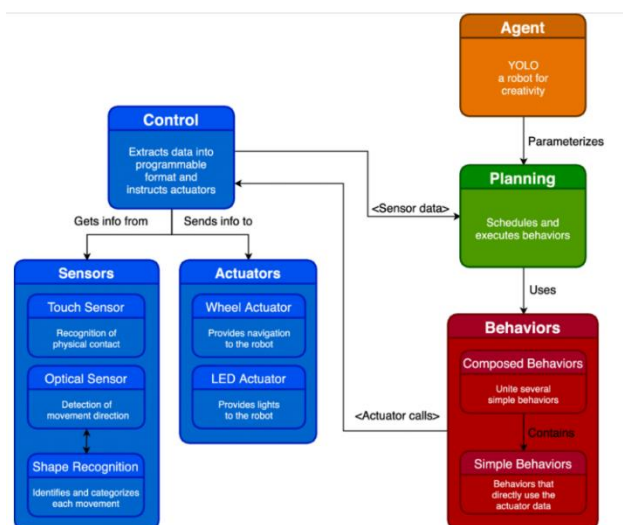
ที่มาของรูป <https://www.collvy.com/blog/controlling-dc-motor-with-raspberry-pi>

2.1.2 โมดูลารไเรเซชันและการแยกส่วนงาน

- ส่วนการตั้งค่า มีหน้าที่จัดการการกำหนดค่าฮาร์ดแวร์เริ่มต้นทั้งหมด โดยเน้นไปที่การควบคุมพอร์ตขาเข้า/ขาออก (GPIO) และการควบคุมความเร็วมอเตอร์ด้วยสัญญาณแบบพัลส์ (PWM)
- ส่วนการควบคุมมอเตอร์ แบ่งเป็น 2 ระดับ ส่วนแรกจะเป็นฟังก์ชันระดับสูง เช่น `forward()`, `backward()`, `left()`, `right()`, และ `stop()` จะทำหน้าที่แปลคำสั่งที่เข้าใจได้ง่ายให้เป็นชุดของคำสั่ง

ควบคุมระดับต่ำซึ่งเป็นส่วนที่สอง ส่วนที่สองฟังก์ชันระดับต่ำเป็นฟังก์ชันภายในที่จะส่งสัญญาณไปให้โมดูลขับเคลื่อนมอเตอร์ เช่น `_drive_motor_a` และ `_drive_motor_b` จะรับผิดชอบการส่งสัญญาณ PWM ควบคุมความเร็วหรือ GPIO

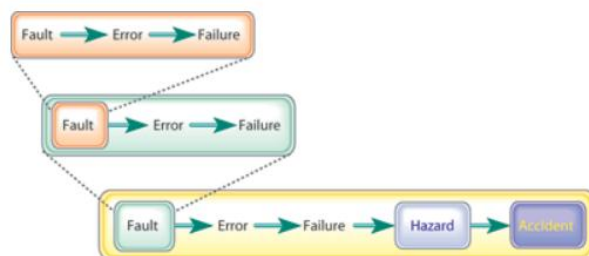
- ส่วนการประมวลผลภาพเป็นการจัดการพารามิเตอร์: ฟังก์ชัน `parse\_args()` มักใช้ไลบรารีอย่าง `argparse` ใน Python เพื่ออ่านพารามิเตอร์การทำงานจากบรรทัดคำสั่ง (Command Line)



ตัวอย่างแผนภาพที่แสดงการแบ่งส่วนงานตามหลักการโมดูลาร์ในระบบหุ่นยนต์

2.1.3 การจัดการทรัพยากรและการปิดระบบอย่างปลอดภัย (Safe Shutdown)

- การทำความสะอาด (Cleanup): ฟังก์ชัน `_safe_shutdown` และบล็อก `finally` ใน `main()` จะเรียกใช้ `pwmA.stop()`, `pwmB.stop()` และ `GPIO.cleanup()` เพื่อคืนค่าพิน GPIO สู่สถานะเริ่มต้น ป้องกันปัญหาไฟฟ้าลัดวงจรหรือพินค้าง

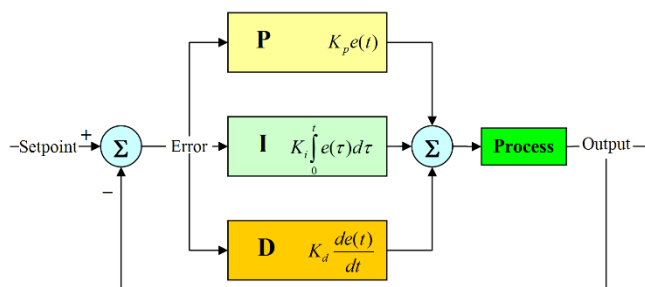


ตัวอย่างหากไม่มี safe shutdown ระบบจะขัดข้องเสียหายได้

ที่มา <https://www.embedded.com/architecture-of-safety-critical-systems/>

2.1.4 การใช้ค่าคงที่และการปรับจูนพฤติกรรม

- มีการกำหนดค่าคงที่ไว้ที่ส่วนบนของโค้ด เช่น SPEED_FWD, SPEED_TURN, TURN_DUR, COMMAND_HOLD ทำให้ง่ายต่อการปรับจูนพฤติกรรมของรถโดยไม่ต้องเข้าไปแก้ไขในลอจิกการทำงานหลัก



เป็นแนวคิดของระบบ PID control

2.2 เทคนิคการประมวลผลภาพและ YOLO (Vision & AI)

2.2.1 การใช้ไลบรารีในการตั้งค่ากล้อง

Pi camera

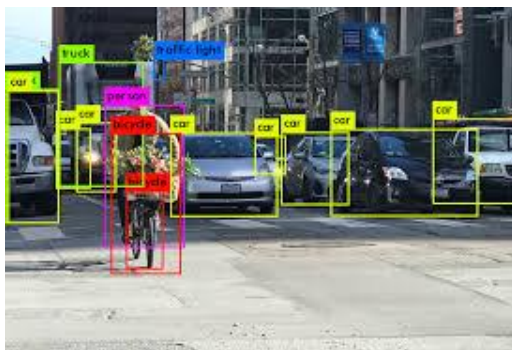
- ใช้ไลบรารี Picamera2 ตั้งค่า create_video_configuration เพื่อกำหนดขนาดภาพ (args.width, args.height) และรูปแบบ ("RGB888") รวมถึงการตั้งค่า Transform สำหรับการหมุนภาพและใช้ picam2.set_controls เพื่อตั้งค่าควบคุมกล้องโดยละเอียด เช่น FrameDurationLimits สำหรับจำกัดอัตราเฟรม

Webcam camera

- ใช้ cv2.VideoCapture พร้อมตั้งค่า cv2.CAP_PROP_ (เช่น FRAME_WIDTH, FRAME_HEIGHT, FPS) เพื่อเข้าถึงและตั้งค่ากล้องมาตรฐาน

2.2.2 การใช้ YOLO สำหรับการตรวจจับวัตถุ (Object Detection)

- ใช้ไลบรารี ultralytics เพื่อโหลดโมเดลจากไฟล์ที่กำหนดผ่าน Argument มีการกำหนดค่า device เพื่อเลือกใช้ CPU หรือ GPU/CUDA



ตัวอย่างของ YOLO ที่ทำการตรวจจับวัตถุ

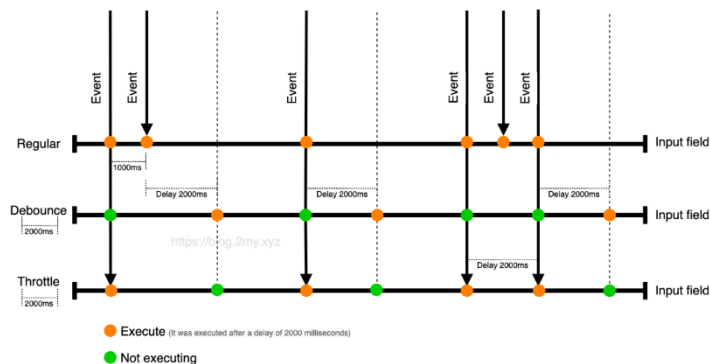
2.2.3 เทคนิคการจัดการเฟรมและการเพิ่มประสิทธิภาพ (Frame Management & Performance)

- เทคนิค Frame Skipping เป็นการใช้ Argument `--skip` และตัวแปร `frame_i` เพื่อเว้นเฟรมในการประมวลผล YOLO ซึ่งเป็นเทคนิคทั่วไปในการเพิ่มความเร็วในการตัดสินใจโดยแลกกับความสามารถในการตรวจสอบจะลดลง ซึ่งอาจทำให้พลาดวัตถุที่เคลื่อนที่เร็วมาก แต่ในบริบทของหุ่นยนต์ที่เคลื่อนที่ช้าหรือตรวจสอบวัตถุที่มีขนาดใหญ่จะแม่นยำ

- การเตรียมโมเดลให้พร้อมทำงานหรือ "Warmup" Model เป็นเทคนิคที่เรียบง่ายแต่มีประโยชน์มาก โดยคือการเมื่อโมเดล Deep Learning ถูกโหลดครั้งแรกเข้าสู่หน่วยความจำการทำงานของ CUDA/GPU Backend มีการประมวลผลเฟรมแรก ๆ ช้ากว่าปกติมาก ทำให้ต้องมีการรันโมเดลด้วยภาพเปล่า 1 ครั้ง ก่อนการเข้าสู่การประมวลผลเฟรมหลัก

2.2.4 การจัดการลอจิกการตัดสินใจ (Decision Logic)

- การป้องกันการสั่งซ้ำถี่เกิน Command Throttling เป็นเทคนิคที่สำคัญเพื่อป้องกันไม่ให้ระบบควบคุมส่งคำสั่งเดียวกันซ้ำๆ อย่างรวดเร็วเกินไปในช่วงเวลาสั้น ซึ่งอาจนำไปสู่ปัญหาด้านเสถียรภาพและประสิทธิภาพของฮาร์ดแวร์ หลักการทำงานของลอจิกการตัดสินใจจะตรวจสอบว่าตั้งแต่ `last_cmd_time` จนถึงเวลาปัจจุบัน ให้ผ่านพ้นช่วง `COMMAND_HOLD` ไปแล้ว หากยังไม่ผ่านระบบจะไม่ส่งคำสั่งเดิมซ้ำ เพื่อป้องกันไม่ให้เกิดสัญญาณรบกวนทางไฟฟ้าที่เกิดจากการเปิด/ปิดอย่างรวดเร็ว ซึ่งอาจจะทำให้โมดูลขับเคลื่อนเสียหายได้



รูปภาพหลักการ Throttling

ที่มา <https://blog.2my.xyz/2022/05/14/what-is-throttling/>

- ลอจิกการเลี้ยวตามเวลา (Timed Turns): คำสั่งเลี้ยว (left, right) ถูกตามด้วย `time.sleep(TURN_DUR)` เพื่อให้รถหมุนตามเวลาที่กำหนด ก่อนจะกลับมาสั่ง `forward(SPEED_FWD)` ทันที เพื่อให้รถออกจากสถานะการเลี้ยวและกลับสู่โหมดการเคลื่อนที่หรือการค้นหาเป้าหมาย

```
kb@phoenixNAP:~$ cat sleep.py
import time
for i in range(3):
    print("Step", i+1)
    time.sleep(1)
kb@phoenixNAP:~$ python3 sleep.py
Step 1
Step 2
Step 3
```

รูปตัวอย่างหลักการ `time.sleep` เป็นการทำให้คำสั่งให้ค้างต่อไปตามระยะเวลาที่กำหนด

2.2.5 ลอจิกการแก้ไขผลลัพธ์การตรวจจับ (Special Detection Check)

- มีการใช้ลอจิกพิเศษเพื่อปรับแก้ผลลัพธ์ของ YOLO หากตรวจเจอ `cls_id == 2` (สมมติว่าเป็นป้าย 'ขวา') โค้ดจะลองรัน YOLO อีกครั้งด้วยค่า `conf` (Confidence Threshold) ที่ต่ำลง (0.1) เพื่อตรวจสอบว่ามี `cls_id == 1` (สมมติว่าเป็นป้าย 'ซ้าย') อยู่ด้วยหรือไม่ เทคนิคนี้อาจใช้เพื่อจัดการความไม่แน่นอนของโมเดลหรือกรณีที่ป้ายมีความคล้ายคลึงกัน

2.2.6 เทคนิคการดีบั๊กและการแสดงผล (Debugging & Visualization)

- การแสดงผลเพื่อดีบั๊ก: ใช้ Argument `--show` ร่วมกับฟังก์ชัน `cv2.imshow` และ `results[0].plot()` เพื่อแสดงผลลัพธ์การตรวจจับ (Bounding Box) และสถานะคำสั่งควบคุมแบบเรียลไทม์

2.3 เทคนิคการทำ Autorun สำหรับสคริปต์ Python บน Raspberry Pi ด้วย systemd

2.3.1 การเตรียมสคริปต์ Python

- เพื่อให้สคริปต์ Python สามารถรันได้โดยตรงโดยไม่ขึ้นอยู่กับสภาพแวดล้อม Shell ต้องมีการเตรียมไฟล์ Shebang: เพิ่มบรรทัด `#!/usr/bin/env python3` ไว้บนสุดของไฟล์ เพื่อระบุ Interpreter ที่ชัดเจน
- สิทธิ Execute : กำหนดสิทธิในการเรียกใช้งาน (+x) ให้กับไฟล์ด้วยคำสั่ง `chmod +x [path_to_file]` เพื่อให้ระบบสามารถสั่งรันไฟล์ได้โดยตรง

2.3.2 การสร้างและกำหนดค่า Service File

- Service File เป็นใบสั่งงาน ที่มอบหมายให้ systemd จัดการการเริ่มต้นและการทำงานของโปรแกรมในพื้นหลัง ตัวอย่างเนื้อหา

[Unit]

Description=YOLO Traffic Sign Robot Control

After=network.target

[Service]

User=boss

WorkingDirectory=/home/boss/

ExecStart=/usr/bin/python3 /home/boss/CAMMMM.py

Restart=on-failure

RestartSec=5s

[Install]

WantedBy=multi-user.target

2.3.3 การเปิดใช้งานและควบคุม Service

- สั่งให้ systemd อ่านและโหลดไฟล์ Service ใหม่ที่สร้างขึ้นมา ใช้คำสั่ง `sudo systemctl daemon-reload`
- ตั้งค่าให้ Service นี้ เริ่มต้นโดยอัตโนมัติ ทุกครั้งที่ Raspberry Pi บูตเสร็จสมบูรณ์ ใช้คำสั่ง `sudo systemctl enable yolo-motor.service`
- เริ่มต้น Service ทันทีเพื่อทดสอบการทำงาน ใช้คำสั่ง `sudo systemctl start yolo-motor.service`
- ตรวจสอบสถานะการทำงาน ใช้คำสั่ง `sudo systemctl status yolo-motor.service` และดูว่ามี active (running)
- ถ้ามีการรีบูตด้วยคำสั่ง `sudo reboot`

2.4 เทคนิคหลักการขับเคลื่อน (Differential Drive Kinematics)

2.4.1 การควบคุมการเคลื่อนที่พื้นฐาน

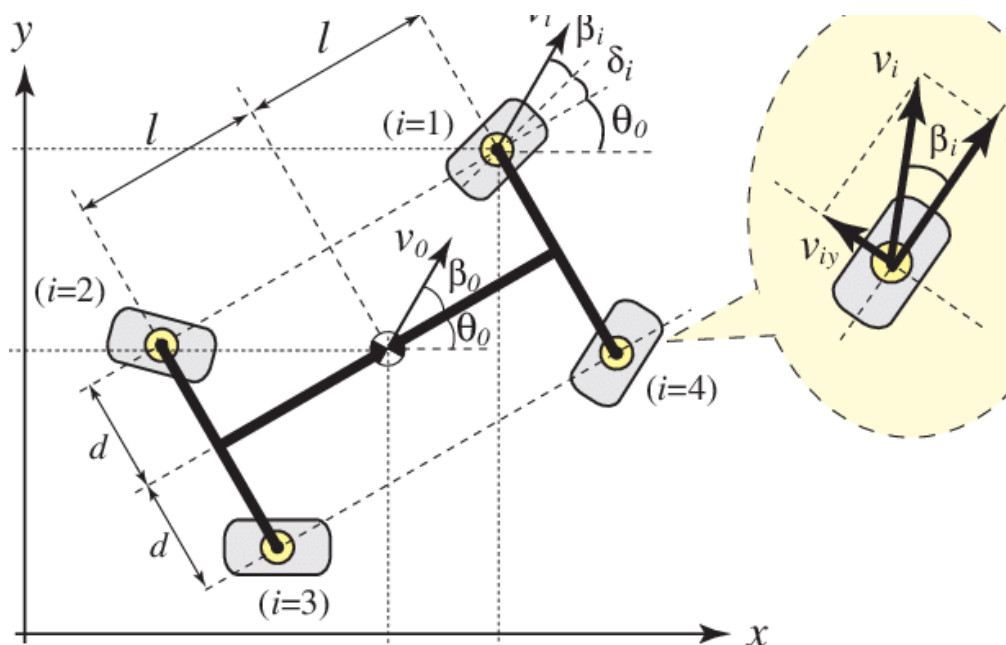
การเคลื่อนที่	เงื่อนไขความเร็ว (V_L, V_R)	ผลลัพธ์เชิงกลศาสตร์
เคลื่อนที่ตรง	$V_L = V_R$	ความเร็วเชิงมุม $\omega = 0$ (ไม่มีการหมุน)
เลี้ยววงกว้าง	V_L ไม่เท่ากับ V_R แต่ V_L และ V_R มากกว่า 0	หุ่นยนต์หมุนไปทางด้านที่มีความเร็วช้ากว่า (เช่น $V_L < V_R$ จะเลี้ยวซ้าย)
หมุนรอบตัวเอง (Pivot)	$V_L = -V_R$	ความเร็วเชิงเส้น $V = 0$ แต่มีความเร็วเชิงมุม ω ไม่เท่ากับ 0

2.4.2 สมการพื้นฐาน (Kinematic Model)

สมการ	คำอธิบาย
ความเร็วเชิงเส้น	$V = V_r + V_l / 2$
ความเร็วเชิงมุม	$\Omega = V_r - V_l / 2$

2.4.3 การประยุกต์ใช้กับระบบ YOLO

- การตัดสินใจด้วย AI : โมเดล YOLO ทำหน้าที่เป็น ระบบรับรู้ (Perception) โดยตรวจจับป้ายจราจร ("เลี้ยวซ้าย", "เลี้ยวขวา", "หยุด")
- การแปลงคำสั่ง : ผลลัพธ์จากการตรวจจับ (เช่น "เลี้ยวขวา") จะถูกแปลงเป็น ค่าความเร็วที่แม่นยำสำหรับล้อแต่ละข้าง ตามหลักการ Kinematic Model
- การควบคุมมอเตอร์ : ใช้ค่า V_l และ V_r เหล่านั้น (ซึ่งถูกแปลงเป็นค่า Duty Cycle ของ PWM) เพื่อสั่งการมอเตอร์แต่ละตัวผ่าน RPi.GPIO



2.5 บอร์ดควบคุม

2.5.1 Raspberry Pi

- เป็นคอมพิวเตอร์บอร์ดเดียวที่ทำงานบนระบบปฏิบัติการ Linux (Raspberry Pi OS) มีหน่วยประมวลผลกลางแบบ Multi-core หน่วยความจำ RAM ขนาดใหญ่ ทำให้มันมีพลังประมวลผลสูงที่สุด สามารถรันไลบรารี Deep Learning ที่ใช้ทรัพยากรมาก เช่น YOLOv8 และ PyTorch ได้อย่างมีประสิทธิภาพ นอกจากนี้ยังรองรับการเชื่อมต่อกล้อง Pi Camera ผ่านอินเทอร์เฟซ CSI ที่มีความเร็วสูงและมีพอร์ตสำหรับเชื่อมต่ออุปกรณ์เสริมและเครือข่ายเต็มรูปแบบ



[Raspberry Pi 5: EVERYTHING you need to know - YouTube](#)

2.5.2 Open MV

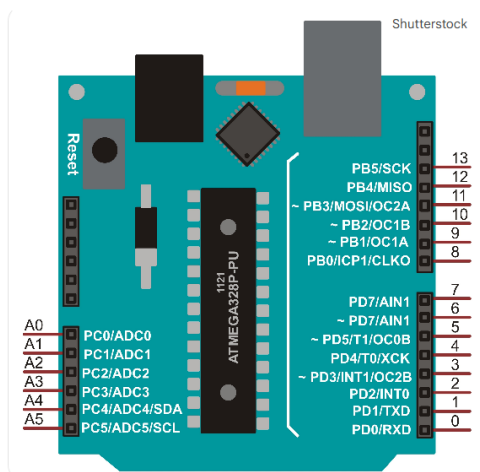
- เป็นบอร์ดไมโครคอนโทรลเลอร์ที่ออกแบบมาโดยเฉพาะสำหรับงาน Machine Vision มีชิปประมวลผล ARM Cortex-M ที่ทรงพลังพอสมควรและรัน MicroPython ได้เหมาะสำหรับงาน Vision ขนาดเล็กถึงกลางที่ต้องการพลังงานต่ำ เช่น การติดตามสี การตรวจจับใบหน้า หรือการรันโมเดล Deep Learning ขนาดเล็กมาก แต่มีข้อจำกัดคือพลังประมวลผลไม่เพียงพอ สำหรับการรันโมเดล Object Detection ที่ซับซ้อนและขนาดใหญ่ เช่น YOLOv8 ที่ใช้ในการตรวจจับป้ายจราจรหลายประเภทในสภาพแวดล้อมที่เปลี่ยนแปลง



<https://openmv.io/pages/openmv-cam-h7>

2.5.3 Arduino

- เป็นไมโครคอนโทรลเลอร์พื้นฐานที่รันโค้ดแบบไม่มี OS และเขียนด้วยภาษา C/C++ เหมาะสำหรับงานควบคุมฮาร์ดแวร์พื้นฐานที่ต้องการความเสถียรของเวลา เช่น การอ่านค่าเซ็นเซอร์, การควบคุมมอเตอร์แบบ PID หรือการควบคุม I/O แต่มีข้อจำกัดไม่สามารถทำการประมวลผลภาพได้เลย เนื่องจากขาดทั้งหน่วยความจำ RAM, หน่วยประมวลผลกลาง, และหน่วยความจำเก็บข้อมูลที่จำเป็นสำหรับการประมวลผลวิดีโอและโมเดล AI



<https://www.youtube.com/watch?v=yW63vW1Y7uM>

2.5.4 ทำไมถึงเลือกใช้บอร์ด Raspberry Pi ในการทำรถเดินตามป้ายจราจร

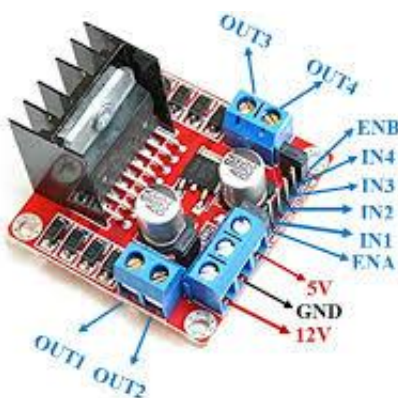
- พลังประมวลผล Deep Learning โมเดล YOLOv8 ต้องการ CPU ที่รวดเร็วและการสนับสนุนไลบรารี AI อย่าง PyTorch/Ultralytics ซึ่ง Raspberry Pi เป็นบอร์ดเดียวในกลุ่มนี้ที่สามารถรันซอฟต์แวร์เหล่านี้ได้อย่างเต็มรูปแบบเพื่อประมวลผลภาพจากกล้อง PiCam (CSI) ได้อย่างรวดเร็ว มีการจัดการระบบที่ซับซ้อนซึ่ง Pi สามารถจัดการงานหลายอย่างพร้อมกัน (Multitasking) ทั้งการรับ Stream ภาพ, การรัน YOLO, การประมวลผลลอจิกการตัดสินใจ, และการสั่งงานมอเตอร์ GPIO/PWM ซึ่งบอร์ดไมโครคอนโทรลเลอร์ไม่สามารถทำได้

2.6 โมดูลขับมอเตอร์

2.6.1 L298N

หมายถึง ไอซีควบคุมมอเตอร์กำลังสูงแบบ Dual H-Bridge รุ่น 298 แพ็กเกจ N

ข้อดี	ข้อเสีย
- ใช้งานง่าย มีโมดูลสำเร็จรูปขายทั่วไป เหมาะสำหรับผู้เริ่มต้นหาง่าย ราคาถูก - ควบคุมได้ 2 มอเตอร์พร้อมกัน ภายในมี 2 H-Bridge แยกกัน (Motor A และ B) ทำให้สามารถควบคุมมอเตอร์ 2 ตัวได้อิสระ รองรับแรงดันสูงถึง 35V ใช้ได้กับมอเตอร์แรงดัน 5V–35V ซึ่งยืดหยุ่นสำหรับโครงงานหุ่นยนต์หลายระดับ - สามารถควบคุมทิศทางและความเร็วได้ใช้พิน IN1–IN4 สำหรับทิศทาง และ ENA/ENB สำหรับ PWM เพื่อควบคุมความเร็ว - โครงสร้างแข็งแรง โมดูลส่วนใหญ่มีฮีทซิงค์และไดโอดป้องกัน ในตัว เหมาะกับงานหุ่นยนต์หรือรถบังคับ	- สูญเสียพลังงานสูงเนื่องจากใช้ทรานซิสเตอร์ BJT ทำให้มีแรงดันตกคร่อมภายใน พลังงานสูญเสียเป็นความร้อนมากเมื่อมอเตอร์กินกระแสสูง - กระแสขั้วสูงสุดจำกัดไม่เพียงพอสำหรับมอเตอร์ใหญ่ เช่นมอเตอร์เกียร์แรงบิดสูง หรือมอเตอร์ที่กินกระแสเริ่มต้นมาก - ต้องใช้อีทซิงค์ช่วยระบายความร้อน เมื่อมอเตอร์กินกระแสเกิน ~1A จะร้อนเร็ว ถ้าไม่ติดฮีทซิงค์อาจไหม้ได้ - ประสิทธิภาพต่ำกว่าไดร์เวอร์รุ่นใหม่ไดร์เวอร์รุ่นใหม่ เช่น L293D, DRV8833, TB6612FNG) ใช้ MOSFET ที่สูญเสียน้อยกว่า ประสิทธิภาพดีกว่า

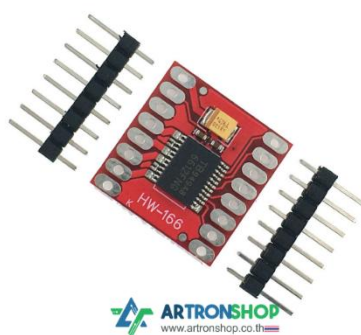


<https://www.flying-tech.com/blog/l298n-motor-driver-ic-features-datasheet-motor-control-applications/>

2.6.2 TB6612FNG

เป็น Motor Driver IC แบบ Dual H-Bridge ผลิตโดยบริษัท Toshiba ทำหน้าที่คล้ายกับ L298N ใช้ควบคุมมอเตอร์ DC ได้ 2 ตัว หรือ Stepper Motor 1 ตัว

ข้อดี	ข้อเสีย
- สูญเสียพลังงานต่ำมาก แรงดันตกคร่อมเพียง $\sim 0.5V$ (เทียบกับ L298N ที่ $\sim 2V$) ทำให้แรงดันที่ส่งถึงมอเตอร์มากขึ้นมอเตอร์หมุนแรงกว่า และไม่ร้อนง่าย เพราะใช้ MOSFET ภายใน - ร้อนน้อยกว่ามาก เนื่องจากมีประสิทธิภาพสูง ($\sim 90-95\%$) จึงแทบไม่ต้องใช้อีทซิงค์ - รองรับ Logic 3.3V และ 5V ใช้ได้ทั้งกับ Arduino, ESP32, ESP8266, Raspberry Pi, STM32 โดยไม่ต้องต่อวงจรแปลงสัญญาณเพิ่ม - ควบคุมง่ายเหมือน L298N แต่ตอบสนองไวกว่า ใช้พิน IN1/IN2 + PWM ควบคุมความเร็วและทิศทางแบบเดียวกัน แต่การตอบสนองของ PWM เร็วกว่ามาก	- แรงดันมอเตอร์จำกัด (ไม่เกิน 13.5V) ไม่เหมาะกับมอเตอร์ที่ใช้แรงดันสูง เช่น 18V หรือ 24V - กระแสต่อเนื่องต่ำกว่า L298N (1.2A เทียบกับ 2A) - ขนาดขาคเล็ก ตะกั่วแน่น (ในเวอร์ชัน IC เปล่า) ถ้าไม่ใช้โมดูลสำเร็จรูป บัดกรียากเพราะเป็นชิป SMD - ไม่มีวงจรแปลงแรงดันในตัว ต้องจ่ายไฟ Vcc (Logic) และ Vm (Motor) แยกกันอย่างถูกต้อง

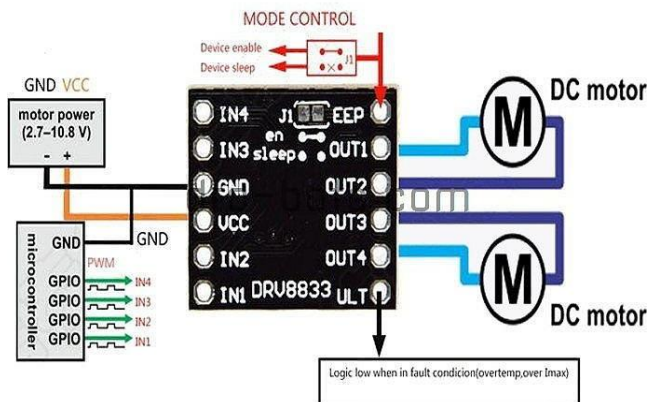


<https://lastminuteengineers.com/tb6612fng-dc-motor-driver-tutorial/>

2.6.3 DRV8833

คือ Dual H-Bridge Motor Driver IC ที่ผลิตโดย Texas Instruments (TI) ใช้สำหรับควบคุม มอเตอร์ DC ได้ 2 ตัว หรือ stopper motor ได้ 1 ตัว ที่ใช้ MOSFET เป็นทรานซิสเตอร์ภายใน

ข้อดี	ข้อเสีย
- ใช้ MOSFET ภายในทำให้มีประสิทธิภาพสูงมาก สูญเสียพลังงานน้อยกว่า L298N อย่างมหาศาล (แรงดันตกคร่อมต่ำ ~0.3–0.5V เท่านั้น) ทำให้มอเตอร์ หมุนแรงกว่าในแรงดันเท่ากัน - รองรับแรงดันลอจิกต่ำสุด 2.7V ใช้ได้กับ ไมโครคอนโทรลเลอร์สมัยใหม่ทุกแบบ เช่น ESP32, Raspberry Pi Pico, STM32, Arduino (5V) โดยไม่ต้องต่อวงจรแปลงสัญญาณ - มี Sleep Mode เมื่อไม่ได้ใช้งานสามารถเข้าสู่โหมด ประหยัดพลังงาน	- แรงดันมอเตอร์ต่ำ (สูงสุด ~10.8V) ไม่เหมาะกับ มอเตอร์ที่ต้องการ 12V เต็ม ๆ หรือสูงกว่า เช่น มอเตอร์เกียร์แรงบิดสูง - กระแสสูงสุดจำกัด (1.5A ต่อช่อง) ถ้ามอเตอร์กิน กระแสมากกว่า 1.5A ต่อเนื่อง อาจต้องใช้ไดร์เวอร์รุ่น ใหญ่กว่า เช่น BTS7960 หรือ L298N - ขาเล็กมากในเวอร์ชันชิป (SMD) ถ้าไม่ได้ใช้โมดูล สำเร็จรูป จะบัดกรีได้ยากมาก - ไม่ทนต่อไฟกระชากแรง ๆ ถ้ามอเตอร์กระตุกหรือ ติดขัดอาจเกิด Overcurrent ได้ (แต่มีระบบตัด ป้องกันอยู่)



<https://www.pololu.com/product/2130>

2.6.4 ทำไมถึงเลือกใช้โมดูลมอเตอร์ L298N ในการทำรถเดินตามป้าย

- เนื่องจาก L298N มีราคาที่ย่อมเยา ใช้งานง่าย เหมาะแก่่มือใหม่ในการเรียนรู้ และสามารถใช้กับมอเตอร์ที่มีแรงดันไฟฟ้าสูงกว่าตัวอย่างที่ยกมาได้ และโดยส่วนใหญ่มาในรูปแบบสำเร็จรูปแล้วโดยที่เราไม่ต้องเพิ่มเติมนะ และโมดูล L298N เป็นชิ้นที่ใหญ่ สะดวกแก่การลงมือทำ และถึงแม้ว่า TB6612FNG และ DRV8833 จะมีประสิทธิภาพทางไฟฟ้าที่ดีกว่ามาก แต่โมดูล L298N ยังคงถูกเลือกใช้ในงานหุ่นยนต์เบื้องต้นที่ใช้ Raspberry Pi ด้วยเหตุผลด้านการเข้าถึงและความพร้อมใช้งาน

2.7 การเปรียบเทียบกล้อง Picam และกล้อง Webcam

2.7.1 การเชื่อมต่อและการใช้ทรัพยากรฮาร์ดแวร์

Picam

- กล้อง Picam ถูกออกแบบมาเฉพาะสำหรับ Raspberry Pi และเชื่อมต่อผ่านพอร์ต CSI ซึ่งเป็นอินเทอร์เฟซฮาร์ดแวร์โดยตรง การเชื่อมต่อลักษณะนี้ทำให้เกิด ความเร็วและความหน่วงต่ำ (Low Latency) ทำให้รถสามารถตอบสนองต่อป้ายจราจรได้ทันที ระบบ Picam และไลบรารี Picamera2 อาศัยฮาร์ดแวร์ ISP (Image Signal Processor) และ GPU ของ Pi ในการจัดการการรับ Stream ภาพและการประมวลผลเบื้องต้น

ทำให้ CPU มีภาระน้อยลงและสามารถท้มเทรพยากรส่วนใหญ่ให้กับงานหนักอย่างการ รันโมเดล YOLO ได้เต็มที่

Webcam

- กล้อง Webcam เชื่อมต่อผ่านพอร์ต USB ซึ่งเป็นอินเทอร์เฟซอเนกประสงค์ ทำให้มีความหน่วงสูงกว่า Picam และมักจะต้องใช้ ทรัพยากร CPU มากกว่าในการประมวลผลข้อมูลและการบีบอัดเฟรมภาพ ซึ่งจะไปแย่งทรัพยากรที่จำเป็นสำหรับโมเดล YOLO โดยตรง ทำให้ประมวลผลช้ากว่า Picam

2.7.2 ผลกระทบต่อระบบ YOLOและการตอบสนอง

Picam

- ด้วยความหน่วงที่ต่ำและภาระ CPU ที่เบาทำให้ Picam เป็นตัวเลือกที่ดีกว่าสำหรับการประยุกต์ใช้สามารถรักษาอัตราเฟรม (FPS) ของกล้องได้สูงอย่างต่อเนื่อง ทำให้ลดโอกาสการตัดสินใจและการควบคุมมอเตอร์เป็นไปอย่างรวดเร็วและแม่นยำ Real-time มากกว่ากล้อง Webcam

Webcam

- ความหน่วงที่สูงกว่าและการใช้ทรัพยากร CPU ที่มากกว่าอาจส่งผลให้ประสิทธิภาพ YOLO ลดลงหาก CPU ต้องรับภาระหนักจากการจัดการภาพพร้อมกัน ประสิทธิภาพในการประมวลผลโมเดล YOLO ก็จะลดลงตามไปด้วย การตอบสนองช้าลง ข้อมูลภาพเข้าสู่การตัดสินใจช้าลง ทำให้การตอบสนองของรถต่อสถานการณ์จริงมีความล่าช้า (Lag)

2.7.3 เทคนิคการจัดการภาพ (Frame Management)

Picam

- ใช้ไลบรารี Picamera2

- ใช้เทคนิคควบคุม FrameDurationLimits เพื่อล็อกอัตราเฟรม ทำให้การ Stream ภาพมีความสม่ำเสมอและควบคุม Latency ได้ดี

Webcam

- ใช้ไลบรารี cv2.VideoCapture

- ใช้เทคนิคควบคุม cv2.CAP_PROP_FPS ซึ่งความแม่นยำในการล็อก FPS และ Latency อาจไม่
สม่ำเสมอเท่า Picamera2



2.8 ทำไมถึงเลือกอัลกอริทึมแบบขั้นตอนเดียวในการตรวจจับวัตถุแบบเรียลไทม์

- ที่ใช้แบบอัลกอริทึมแบบขั้นตอนเดียวแทนที่จะใช้อัลกอริทึมแบบสองขั้นตอน ก็เพราะว่าต้องการให้มองภาพเพียงครั้งเดียวและประมวลทุกอย่างในภาพออกมาได้เลย

2.9 อัลกอริทึมแบบขั้นตอนเดียว

2.9.1 YOLOv3

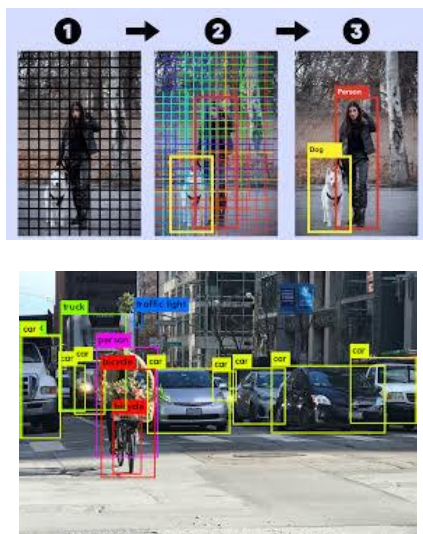
- เป็นอัลกอริทึมตรวจจับวัตถุ (Object Detection Algorithm) ที่ถูกพัฒนาต่อจาก YOLOv1 และ YOLOv2 โดย Joseph Redmon ในปี 2018 จุดเด่นคือ ตรวจจับวัตถุในภาพได้อย่างรวดเร็วและแม่นยำ โดยประมวลผลภาพ เพียงครั้งเดียว (one pass) แทนการสแกนหลายจุดแบบเก่า

2.9.2 YOLOv7

- ถูกพัฒนาต่อจาก YOLOv3, YOLOv4 และ YOLOv5 ถูกเปิดตัวโดย WongKinYiu ในปี 2022 จุดเด่นคือ แม่นยำสูงขึ้นและเร็วกว่า YOLO รุ่นก่อน ๆ เน้นประสิทธิภาพกับ วัตถุขนาดเล็ก และ งาน real-time detection

2.9.3 YOLOv8

- ถูกพัฒนาโดย Ultralytics ในปี 2023 เป็นรุ่นล่าสุดของตระกูล YOLO จุดเด่นคือ แม่นยำที่สุด, ใช้งานง่าย, รองรับงานหลายประเภทในโมเดลเดียว รองรับ Detection, Segmentation, Classification, Pose Detection



ตารางการเปรียบเทียบข้อดี ข้อเสียของแต่ละเวอร์ชัน

อัลกอริทึม ตรวจจับ วัตถุ	ข้อดี	ข้อเสีย
YOLOv3	- แม่นยำสำหรับวัตถุกลาง-ใหญ่ - ตรวจจับแบบ multi-scale - Real-time detection (30 FPS บน GPU ปานกลาง) - ใช้เป็นมาตรฐานเรียนรู้ Object Detection	- ขนาดโมเดลใหญ่ (Darknet-53) จะใช้memory เยอะ - การตรวจจับวัตถุเล็กไม่แม่นยำ - ไม่มี segmentation/pose detection - ปรับแต่งยากกว่า YOLOv5-v8

YOLOv7	- AP สูงกว่า YOLOv3/4 - ตรวจจับวัตถุเล็กแม่นยำกว่า - Real-time detection 40-50 FPS - ปรับ model size ได้ตามhardware	- ขนาดโมเดลใหญ่สำหรับ -YOLOv7-X ต้องใช้ GPU ที่สเปกสูง - ไม่มี segmentation/pose detection ในตัว - การตั้งค่า config และ hyperparameter ซับซ้อน - ใช้ PyTorch แต่ไม่มี official Ultralytics support
YOLOv8	- แม่นยำที่สุดในปัจจุบัน (AP สูงกว่า YOLOv7 เล็กน้อย) - รองรับ multi-task: detection + segmentation + pose - Real-time detection 50–60+ FPS - โมเดลยืดหยุ่น: สามารถเลือกตาม GPU - ใช้งานง่ายด้วย PyTorch + Ultralytics Hub Export ง่าย: TorchScript, ONNX, TensorRT, CoreML	- เป็นอัลกอริทึมตัวใหม่มาก ก็เลยมีเอกสาร หรือคู่มือการเรียนรู้ที่น้อย - ต้องใช้ PyTorch 1.13+ - dataset บางอย่างต้องปรับ Hyperparameters

2.9.4 ทำไมถึงเลือกใช้ YOLOv8 ในการทำรถเดินตามป้าย

- เพราะว่า YOLOv8 เป็นอัลกอริทึมตัวใหม่ล่าสุดที่ให้ผลประมวลเร็วที่สุดอยู่ที่ 50-60+ FPS และมีความแม่นยำสูงที่สุดเมื่อเทียบกับรุ่นอื่น โมเดลมีขนาดยืดหยุ่นกว่ารุ่นอื่น มีความง่ายต่อการใช้งานมากกว่ารุ่นอื่นและสามารถรองรับโปรเจกเล็กๆ ได้ดีกว่า

2.10 งานวิจัยและพัฒนาที่คล้ายคลึงกับโครงการรถเดินตามป้าย

2.10.1 โครงการการตรวจจับป้ายจราจรด้วย YOLOv8 บน Raspberry Pi

สาระสำคัญและวิธีการ

- เป้าหมาย พัฒนาระบบตรวจจับและจดจำป้ายจราจรของอินเดีย (Indian traffic signs) แบบ Real-time และให้เสียงเตือนแก่ผู้ขับขี่
- ฮาร์ดแวร์หลัก Raspberry Pi 4 (4GB RAM) และ Raspberry Pi Camera Module (PiCam)
- เทคนิค/โมเดล โมเดล YOLOv8 ในการตรวจจับวัตถุ (Object Detection) โดยฝึกโมเดลด้วยชุดข้อมูลป้ายจราจรที่สร้างขึ้นเอง (In-house-developed dataset)
- การนำไปใช้งาน ติดตั้งโมเดล YOLOv8 ที่ฝึกแล้วบน Raspberry Pi และใช้ไลบรารี OpenCV ร่วมกับ pyttsx3 (Text-to-Speech) เพื่อให้เสียงแจ้งเตือนตามป้ายที่ตรวจพบ (เช่น "STOP," "SPEED LIMIT")
- ผลลัพธ์ โมเดลแสดงประสิทธิภาพที่ดีและมีความรวดเร็วในการอนุมาน (Fast Inference) ซึ่งพิสูจน์ว่า YOLOv8 เหมาะสำหรับการใช้งานแบบ Real-time บนฮาร์ดแวร์ที่มีทรัพยากรจำกัดอย่าง Raspberry Pi

ความเกี่ยวข้องกับโครงการรถเดินตามป้าย

- มีการเลือกใช้ YOLOv8 และ Raspberry Pi และ PiCam ในการตรวจจับวัตถุแบบ Real-time ซึ่งเป็นหัวใจหลักของรถเดินตามป้าย

อ้างอิง

Road Sign Detection System using YOLOv8 on Raspberry Pi 4

<https://ijrpr.com/uploads/V6ISSUE4/IJPR42880.pdf>

2.10.2 โครงการหุ่นยนต์ตามมนุษย์โดยใช้ YOLOv5 และสถาปัตยกรรมแบบกระจาย

สาระสำคัญและวิธีการ

- เป้าหมาย พัฒนารถหุ่นยนต์ให้สามารถติดตามและเดินตามมนุษย์ได้แบบ Real-time
- ฮาร์ดแวร์หลัก ชุดรถหุ่นยนต์ Raspberry Pi 4
- เทคนิค/โมเดล โมเดล YOLOv5 ในการตรวจจับ "คน" (Person Detection)
- การออกแบบระบบ (Distributed Architecture) เนื่องจาก YOLOv5 ต้องการพลังการประมวลผลสูง งานนี้ได้แก้ปัญหาทรัพยากรจำกัดของ Raspberry Pi โดยใช้ สถาปัตยกรรมแบบกระจาย (Distributed Architecture)
- Raspberry Pi ทำหน้าที่ควบคุมการเคลื่อนที่ของหุ่นยนต์ และรับข้อมูลภาพจากกล้อง External Laptop (พร้อม GPU) ทำหน้าที่ ประมวลผล Object Detection (Inference) โดยรับภาพจาก Pi และส่งพิกัดวัตถุกลับมาทางเครือข่าย ROS
- ผลลัพธ์ สามารถติดตามมนุษย์ได้อย่างราบรื่นแบบ Real-time โดยใช้ประโยชน์จากพลัง GPU ภายนอก ในการคำนวณที่หนักหน่วง

ความเกี่ยวข้องกับโครงการรถเดินตามป้าย

- งานนี้แสดงให้เห็นถึง วิธีการแก้ปัญหาด้านประสิทธิภาพ ในโครงการหุ่นยนต์ที่ใช้ Deep Learning บน Raspberry Pi
- ความท้าทาย โครงการรถเดินตามป้ายต้องมีการตรวจจับแบบ Real-time หากพบว่า YOLOv8 ยังเข้าเกินไป บน Pi รุ่นเล็ก โครงการนี้เสนอแนวคิดที่สามารถใช้ "External Computation" (เช่น โน้ตบุ๊ก, คอมพิวเตอร์ตั้งโต๊ะ) มาช่วยประมวลผลภาพ แล้วส่งผลลัพธ์การตรวจจับกลับมาให้ Raspberry Pi ทำหน้าที่แค่ "ควบคุมการเคลื่อนที่" เท่านั้น

อ้างอิง

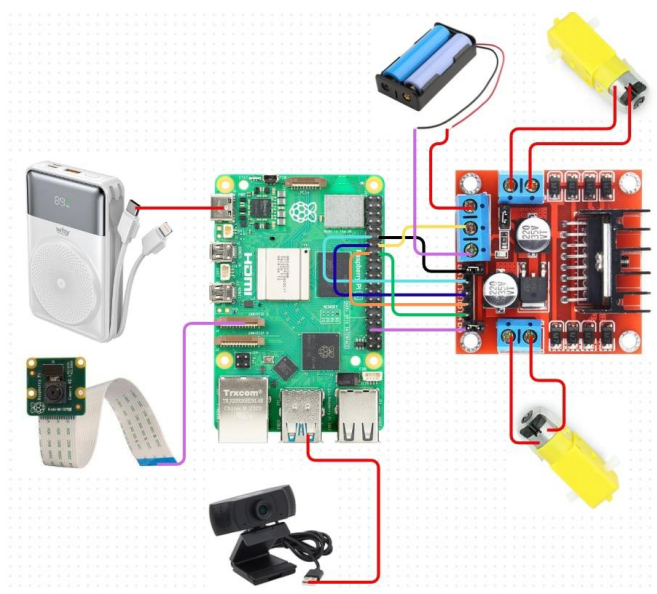
Human Following algorithm implemented on the Adept AWR 4WD WiFi Smart Robot Car Kit for Raspberry Pi 4 Model. Utilizes YOLOv5 for person detection, empowering the robot to track and follow a human

<https://github.com/khalidbourr/Adept-RaspberryYOLO-Follower>

บทที่ 3

หลักการและแนวคิดในการออกแบบระบบ

เป็นระบบที่สามารถตรวจจับและจดจำป้ายจราจร เช่น ซ้าย, ขวา, เดินหน้าและหยุด จากภาพที่ได้จากกล้องแล้วนำผลการตรวจจับนั้นไปใช้ในการควบคุมการเคลื่อนที่ของรถให้สอดคล้องกับคำสั่งบนป้ายโดยอัตโนมัติ หลักการทำงานของระบบโดยทั่วไปประกอบด้วย 4 ขั้นตอนหลัก ได้แก่ การรับข้อมูลภาพ การประมวลผลภาพ และการตรวจจับป้าย การตัดสินใจและควบคุมทิศทาง การเคลื่อนที่ของรถ แนวคิดในการออกแบบคือการผสมผสานระหว่างเทคโนโลยีปัญญาประดิษฐ์กับระบบควบคุมทางกลเพื่อสร้างระบบที่สามารถเรียนรู้และตอบสนองต่อสิ่งแวดล้อมได้แบบอัตโนมัติ ระบบจะทำงานแบบ Real-Time โดยอาศัยการประมวลผลภาพจากกล้องเป็นข้อมูลนำทาง



3.1 องค์ประกอบหลักของโครงการ

- 1 Raspberry ทำหน้าที่ประมวลผลและความคุม GPIO และ PWM และการรัน YOLO เป็นหัวใจของชิ้นงานเพราะทำการควบคุมทั้งหมด
- 2 Camera module ทำหน้าที่รับภาพ แปลงข้อมูลภาพเบื้องต้น และส่งข้อมูลไปให้ บอร์ด RPI

3 Motor driver ทำหน้าที่ในการแปลงสัญญาณที่ได้รับจาก Raspberry pi มาเป็นกำลังในการสั่งการขับเคลื่อนให้เคลื่อนที่ตามคำสั่ง

4 DC motors ทำหน้าที่ รับคำสั่งจาก Motor driver และทำการขับเคลื่อนล้อตามคำสั่งที่ได้รับมา

3.2 องค์ประกอบ Vision Layer ของโครงงาน

1 Motor Control Logic เป็นการควบคุมมอเตอร์ในทิศทางต่างๆ และความคุมความเร็วผ่าน PWM

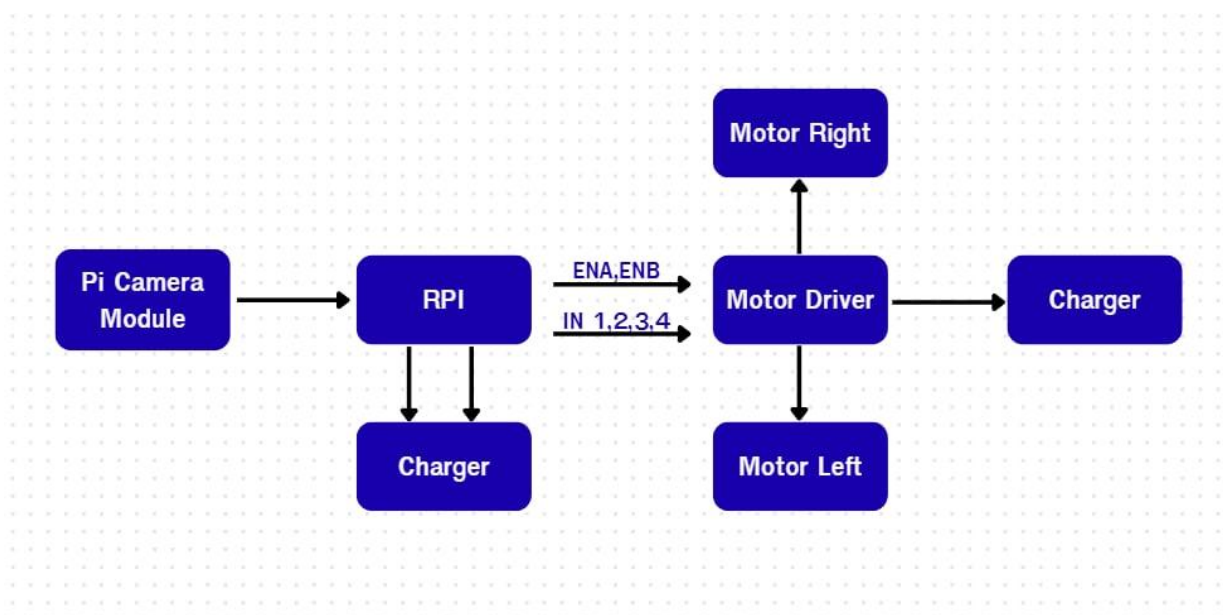
2 Video Capture ทำหน้าที่ดึงเฟรมภาพมาจากกล้อง

3 YOLO มีหน้าที่ในการตัดสินใจ / ตรวจจับวัตถุ เช่น ป้ายจราจร หรือสิ่งอื่นๆที่เราเทรนมา เพื่อระบุคำสั่งที่ต้องทำ

4 Command Logic มีหน้าที่ป้องกันคำสั่งถี่เกินไป โดยอ้างอิงมาจาก YOLO

3.3 ภาพรวมการทำงาน

บอร์ด Raspberry Pi รับภาพจากกล้อง ประมวลผลภาพด้วย YOLO ที่ทำการเทรนมาแล้ว เพื่อตรวจจับวัตถุและตัดสินใจ จากนั้นส่งสัญญาณ PWM GPIO ขาต่างๆไปยัง L298N ซึ่งควบคุมการทำงานของมอเตอร์ให้รถเคลื่อนที่ตามลอจิกที่กำหนด



3.4 ตารางการต่อขา

PIN	GPIO	หน้าที่	ประเภทสัญญาณ
ENA	18	ควบคุมความเร็ว Motor A	PWM OUTPUT
IN1	17	กำหนดทิศทาง 1 Motor A	DIGITAL OUTPUT
IN2	27	กำหนดทิศทาง 2 Motor A	DIGITAL OUTPUT
IN3	22	กำหนดทิศทาง 1 Motor B	DIGITAL OUTPUT
IN4	23	กำหนดทิศทาง 2 Motor B	DIGITAL OUTPUT
ENB	19	ควบคุมความเร็ว Motor B	PWM OUTPUT

3.5 ตารางวิธีการเคลื่อนที่และเคลื่อนที่ยังไง

การเคลื่อนที่	Motor A	Motor B	การควบคุมความเร็ว
เดินหน้า	IN1=LOW, IN2=HIGH	IN3=HIGH, IN4=LOW	ENA, ENB = SPEED_FWD
เลี้ยวซ้าย	IN1=HIGH, IN2=LOW	IN3=HIGH, IN4=LOW	ENA, ENB = SPEED_TURN
เลี้ยวขวา	IN1=LOW, IN2=HIGH	IN3=LOW, IN4=HIGH	ENA, ENB = SPEED_TURN
หยุด	IN1=HIGH, IN2=HIGH	IN3=HIGH, IN4=HIGH	ENA, ENB = 0

IN1	IN2	ทิศทางการไหลของกระแส	ทิศทาง
HIGH	HIGH	ไม่มีกระแสไหลผ่าน	แรงดันไฟฟ้าสูงเท่ากัน ทำให้เกิดการต้านและมอเตอร์หยุด

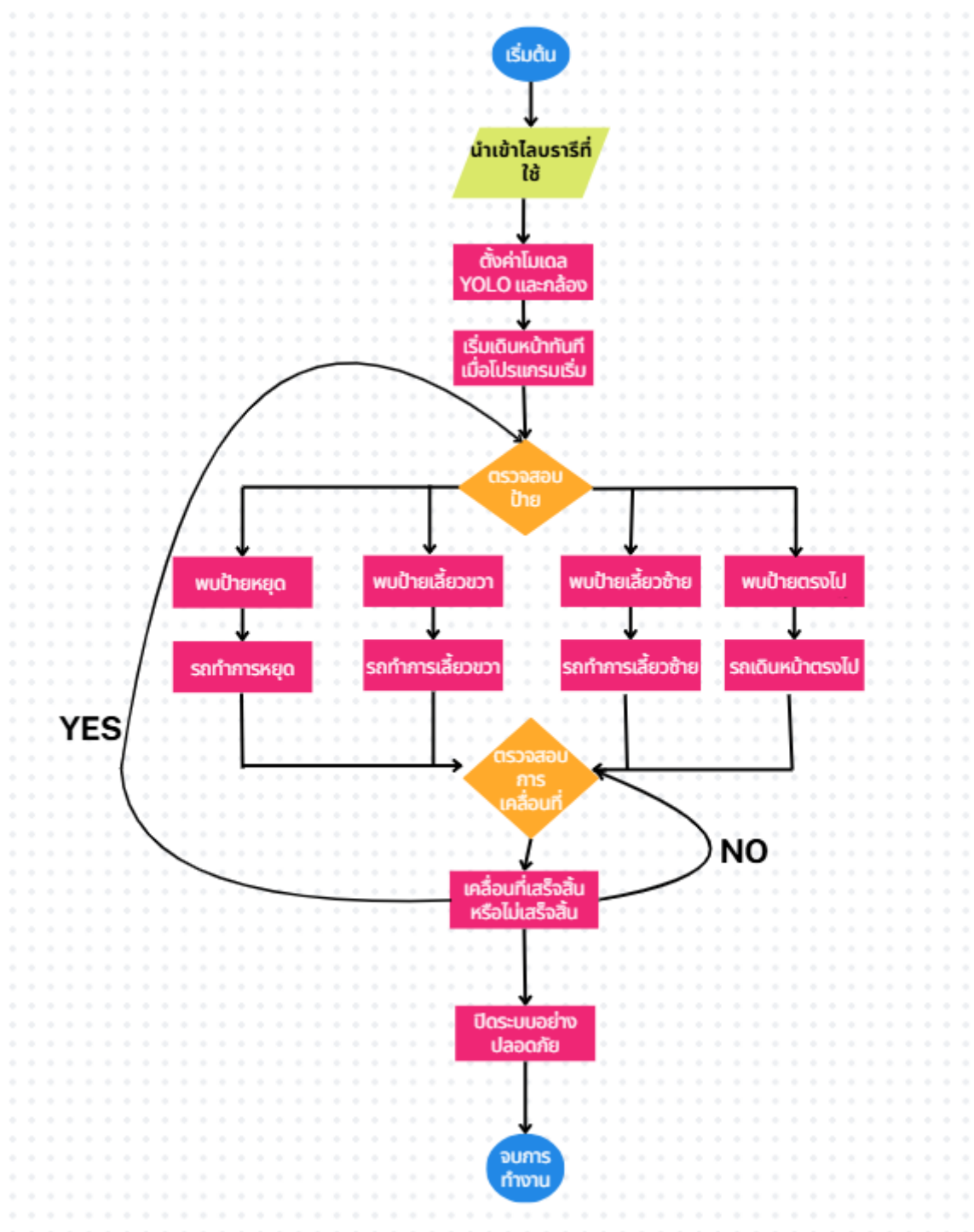
LOW	LOW	ไม่มีกระแสไหลผ่าน	ขาทั้ง 2 ข้างต่อกับ ground เท่ากัน ล้อหมุนช้าๆ (ปลดปล่อยไหล)
HIGH	LOW	ไหลจาก IN1 ไป IN2	กระแสไฟฟ้าหมุนย้อนกลับมอเตอร์หมุนย้อนกลับ
LOW	HIGH	ไหลจาก IN2 ไป IN1	กระแสไฟฟ้าไหลไปข้างหน้า มอเตอร์หมุนไปข้างหน้า

IN3	IN4	ทิศทางการไหลของกระแส	ทิศทาง
HIGH	HIGH	ไม่มีกระแสไหลผ่าน	แรงดันไฟฟ้าสูงเท่ากัน ทำให้เกิดการต้านและมอเตอร์หยุด
LOW	LOW	ไม่มีกระแสไหลผ่าน	ขาทั้ง 2 ข้างต่อกับ ground เท่ากัน ล้อหมุนช้าๆ (ปลดปล่อยไหล)
HIGH	LOW	กระแสไหลจาก IN3 ไป IN4	กระแสไฟฟ้าไหลไปด้านหน้า ทำให้มอเตอร์หมุนไปข้างหน้า
LOW	HIGH	กระแสไหลจาก IN4 ไป IN3	กระแสไฟฟ้าไหลกลับกัน ทำให้มอเตอร์หมุนย้อนกลับ

3.6 สรุปการเคลื่อนที่

- การเลี้ยวขวาคือการที่ล้อทางด้านซ้าย IN1 และ IN2 หมุนไปด้านหน้าและล้อทางด้านขวา IN3 และ IN4 หมุนไปด้านหลังเลี้ยวซ้ายคือ การที่ล้อทางด้านซ้าย IN1 และ IN2 หมุนไปถอยหลังและล้อทางด้านขวา IN3 และ IN4 หมุนไปข้างหน้าส่วนการเดินหน้า เป็นการที่ IN1 IN2 IN3 IN4 ไหลไปข้างหน้า ส่วนหยุด IN1 IN2 IN3 IN4 เกิดการต้านกันของแรงดันไฟฟ้าไม่มีการไหลของกระแสไฟ ทำให้ล้อหยุดหมุน

3.7 Pi cam flowchart



3.8 สรุปโครงสร้างโค้ด Picam

1 ส่วนตั้งค่าฮาร์ดแวร์และการเตรียมพร้อม

ส่วนแรก เป็นการนำเข้าไลบรารีที่จำเป็นและการกำหนดค่าคงที่ เช่น หมายเลขขา GPIO สำหรับควบคุมมอเตอร์ (IN1–IN4, ENA, ENB) และการตั้งค่าพื้นฐานของหุ่นยนต์ เช่น SPEED_FWD, TURN_DUR (เวลานานในการเลี้ยว), และ COMMAND_HOLD (เวลายกคันส่งซ้ำถี่เกิน)

```
# testmotor_drive.py
import RPi.GPIO as GPIO
import time
import atexit

# detect_realtime_yolo_picam2.py
import argparse, cv2, torch
from ultralytics import YOLO
from picamera2 import Picamera2
from libcamera import Transform

IN1, IN2 = 17, 27 # Motor A
IN3, IN4 = 22, 23 # Motor B
ENA, ENB = 18, 19 # PWM pins
PWM_FREQ = 1000

A_INVERT = False
B_INVERT = False

pwmA = None
pwmB = None

# ปรับพารามิเตอร์ได้ตรงนี้
SPEED_FWD = 60 # ความเร็วเดินหน้า
SPEED_TURN = 60 # ความเร็วตอนเลี้ยว (pivot)
TURN_DUR = 2 # เวลาหมุนเลี้ยว (วินาที)
COMMAND_HOLD = 1 # เวลายกคันส่งซ้ำถี่เกิน (วินาที)

def _clamp_speed(p):
    return max(0, min(100, int(p)))
```

```
def setup():
    global pwmA, pwmB
    GPIO.setmode(GPIO.BCM)
    GPIO.setup([IN1, IN2, IN3, IN4, ENA, ENB], GPIO.OUT)
    GPIO.output([IN1, IN2, IN3, IN4], GPIO.LOW)
    pwmA = GPIO.PWM(ENA, PWM_FREQ)
    pwmB = GPIO.PWM(ENB, PWM_FREQ)
    pwmA.start(0)
    pwmB.start(0)
    atexit.register(_safe_shutdown)
```

ส่วนสองฟังก์ชัน setup() จะเริ่มต้นขา GPIO, สร้างและเริ่ม PWM Object สำหรับควบคุมความเร็ว

ส่วนสามการป้องกัน ฟังก์ชัน `_safe_shutdown()` เพื่อให้แน่ใจว่าทรัพยากรทั้งหมด (มอเตอร์, กล้อง, GPIO) จะถูกปิดอย่างปลอดภัยเมื่อโปรแกรมจบลง

```
def _safe_shutdown():
    global pwmA, pwmB
    try:
        if pwmA: pwmA.stop()
    except Exception as e:
        print("pwmA.stop() error:", e)
    try:
        if pwmB: pwmB.stop()
    except Exception as e:
        print("pwmB.stop() error:", e)
    try: del pwmA
    except Exception: pass
    try: del pwmB
    except Exception: pass
    time.sleep(0.05)
    try:
        GPIO.cleanup()
    except Exception as e:
        print("GPIO.cleanup() error:", e)
```

2 ส่วนการควบคุมการขับเคลื่อน

```
def _drive_motor_a(direction: int, speed: int):
    """
    direction: 1=forward, -1=backward, 0=stop
    speed: 0..100
    """
    s = _clamp_speed(speed)
    d = direction
    if A_INVERT:
        d = -d
    if d > 0:
        GPIO.output(IN1, GPIO.HIGH); GPIO.output(IN2, GPIO.LOW)
    elif d < 0:
        GPIO.output(IN1, GPIO.LOW); GPIO.output(IN2, GPIO.HIGH)
    else:
        GPIO.output(IN1, GPIO.LOW); GPIO.output(IN2, GPIO.LOW)
    pwmA.ChangeDutyCycle(s if d != 0 else 0)

def _drive_motor_b(direction: int, speed: int):
    s = _clamp_speed(speed)
    d = direction
    if B_INVERT:
        d = -d
    if d > 0:
        GPIO.output(IN3, GPIO.HIGH); GPIO.output(IN4, GPIO.LOW)
    elif d < 0:
        GPIO.output(IN3, GPIO.LOW); GPIO.output(IN4, GPIO.HIGH)
    else:
        GPIO.output(IN3, GPIO.LOW); GPIO.output(IN4, GPIO.LOW)
    pwmB.ChangeDutyCycle(s if d != 0 else 0)
```

ส่วนแรกฟังก์ชันการเคลื่อนที่ระดับต่ำ `_drive_motor_a/b` จัดการสัญญาณทิศทาง (HIGH/LOW) และความเร็วผ่าน PWM.

ส่วนสองฟังก์ชันควบคุมการเคลื่อนที่ระดับสูง เช่น forward(), backward(), left(), และ right() จะสั่งงานมอเตอร์คู่เพื่อสร้างรูปแบบการเคลื่อนที่ที่ต้องการ เช่น การเลี้ยวแบบ Pivot (หมุนกับที่)

```
# ----- ทิศทางตามที่คุณใช้อยู่ -----
def forward(speed=70):
    _drive_motor_a(-1, speed)
    _drive_motor_b(1, speed)

def backward(speed=70):
    _drive_motor_a(1, speed)
    _drive_motor_b(-1, speed)

def left(speed=70):
    _drive_motor_a(1, speed)
    _drive_motor_b(1, speed)

def right(speed=70):
    _drive_motor_a(-1, speed)
    _drive_motor_b(-1, speed)

def stop(brake=False):
    if brake:
        GPIO.output([IN1, IN2, IN3, IN4], GPIO.HIGH)
    else:
        GPIO.output([IN1, IN2, IN3, IN4], GPIO.LOW)
    pwmA.ChangeDutyCycle(0)
    pwmB.ChangeDutyCycle(0)
```

3. ส่วนการเตรียม Vision/AI และ Warmup def main()

```
def main():
    args = parse_args()

    # YOLO model
    model = YOLO(args.model)
    device = args.device
    if device != "cpu" and not torch.cuda.is_available():
        device = "cpu"
    model.to(device)

    # Camera
    picam2 = Picamera2()
    transform = Transform(hflip=False, vflip=False, rotation=args.rotate)
    video_config = picam2.create_video_configuration(
        main={"size": (args.width, args.height), "format": "RGB888"},
        transform=transform,
        buffer_count=2,
    )
    picam2.configure(video_config)
    picam2.set_controls({
        "FrameDurationLimits": (int(1e6/args.fps), int(1e6/args.fps)),
        "AeEnable": True,
        "AwbEnable": True,
    })
    picam2.start()
    time.sleep(0.2)

    # Warmup YOLO
    import numpy as np
    _ = model(np.zeros((args.height, args.width, 3), dtype=np.uint8),
              imgs=args.imgsz, conf=args.conf, verbose=False)
```

อยู่ในฟังก์ชัน main() ส่วนแรกมีการเตรียมการสำหรับระบบ Vision AI โปรแกรมจะโหลดโมเดล YOLOv8 ที่เทรนแล้ว (best.pt) และตั้งค่า PiCam2 โดยมีการกำหนดค่า video_config และใช้ Transform สำหรับการหมุนภาพ (ถ้าจำเป็น) และตั้งค่า FrameDurationLimits เพื่อคุมอัตราเฟรม (FPS). จากนั้นจึงดำเนินการส่วนที่สอง Warmup Model ด้วยภาพเปล่าหนึ่งครั้งเพื่อให้โมเดลพร้อมใช้งานทันทีที่เข้าสู่การ ทำงานหลัก

4. ส่วนลอจิกการทำงานหลักและการตัดสินใจ loop while true

```
try:
    while True:
        rgb = picam2.capture_array()
        do_infer = (args.skip == 0) or (frame_i % (args.skip + 1) == 0)

        if do_infer:
            bgr = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)
            results = model(bgr, imsz=args.imsz, conf=args.conf, verbose=False)

            # คำเริ่มต้น: ถ้าไม่เจออะไร ให้ go ต่อ
            action_name = "go"

            if len(results[0].boxes) > 0:
                # เอากล่องแรกพอสำหรับงานนี้ (เร็ว)
                box = results[0].boxes[0]
                cls_id = int(box.cls)

                # ----- เช็คพิเศษ: ถ้าเป็น 2 ลอง conf ต่ำกว่าเป็น 1 ใต้ไหม -----
                if cls_id == 2:
                    low_conf = 0.1
                    check_results = model(bgr, imsz=args.imsz, conf=low_conf, verbose=False)
                    if any(int(b.cls) == 1 for b in check_results[0].boxes):
                        cls_id = 1

                # map id -> ชื่อ (ถ้าโมเดลมี names ก็ใช้ของโมเดล)
                names = results[0].names
                if cls_id in names:
                    (variable) cls_id: int
                    action_name = names[cls_id]
                else:
                    action_name = {0:"stop", 1:"left", 2:"right", 3:"go"}.get(cls_id, "go")
```

รับภาพจากกล้องและใช้ลอจิก Frame Skipping เพื่อลดภาระการประมวลผล YOLO โดยการเว้นเฟรมตามพารามิเตอร์ `-skip` หากมีการตรวจพบวัตถุโปรแกรมจะใช้กล้องแรกสุด ในการตัดสินใจลอจิกพิเศษถูกใช้เพื่อแก้ไขผลลัพธ์ในสถานการณ์ที่ป้ายจราจรมีความคลุมเครือ โดยหากตรวจพบ `cls_id = 2` (Right) จะทำการรัน YOLO ซ้ำด้วย Confidence ต่ำ (0.1) เพื่อตรวจสอบว่ามี `cls_id = 1` (Left) อยู่ด้วยหรือไม่

```
now = time.time()
if last_cmd_name == action_name and (now - last_cmd_time) < COMMAND_HOLD:
    pass
else:
    # ---- ทำตามป้าย ----
    if action_name == "stop":
        print("stop")
        stop(brake=True)
        # หมายเหตุ: จะหยุดค้างไว้จนกว่าจะเจอป้ายใหม่ (เช่น go/left/right)

    elif action_name == "go":
        print("go")
        forward(SPEED_FWD)

    elif action_name == "left":
        print("left")
        left(SPEED_TURN)
        time.sleep(TURN_DUR)
        forward(SPEED_FWD) # กลับมาเดินหน้าต่อ

    elif action_name == "right":
        print("right")
        right(SPEED_TURN)
        time.sleep(TURN_DUR)
        forward(SPEED_FWD) # กลับมาเดินหน้าต่อ

    else:
        # ถ้า label แปลก ๆ ให้เดินหน้าต่อ
        print(f"unknown '{action_name}' -> go")
        forward(SPEED_FWD)

    last_cmd_name = action_name
    last_cmd_time = now

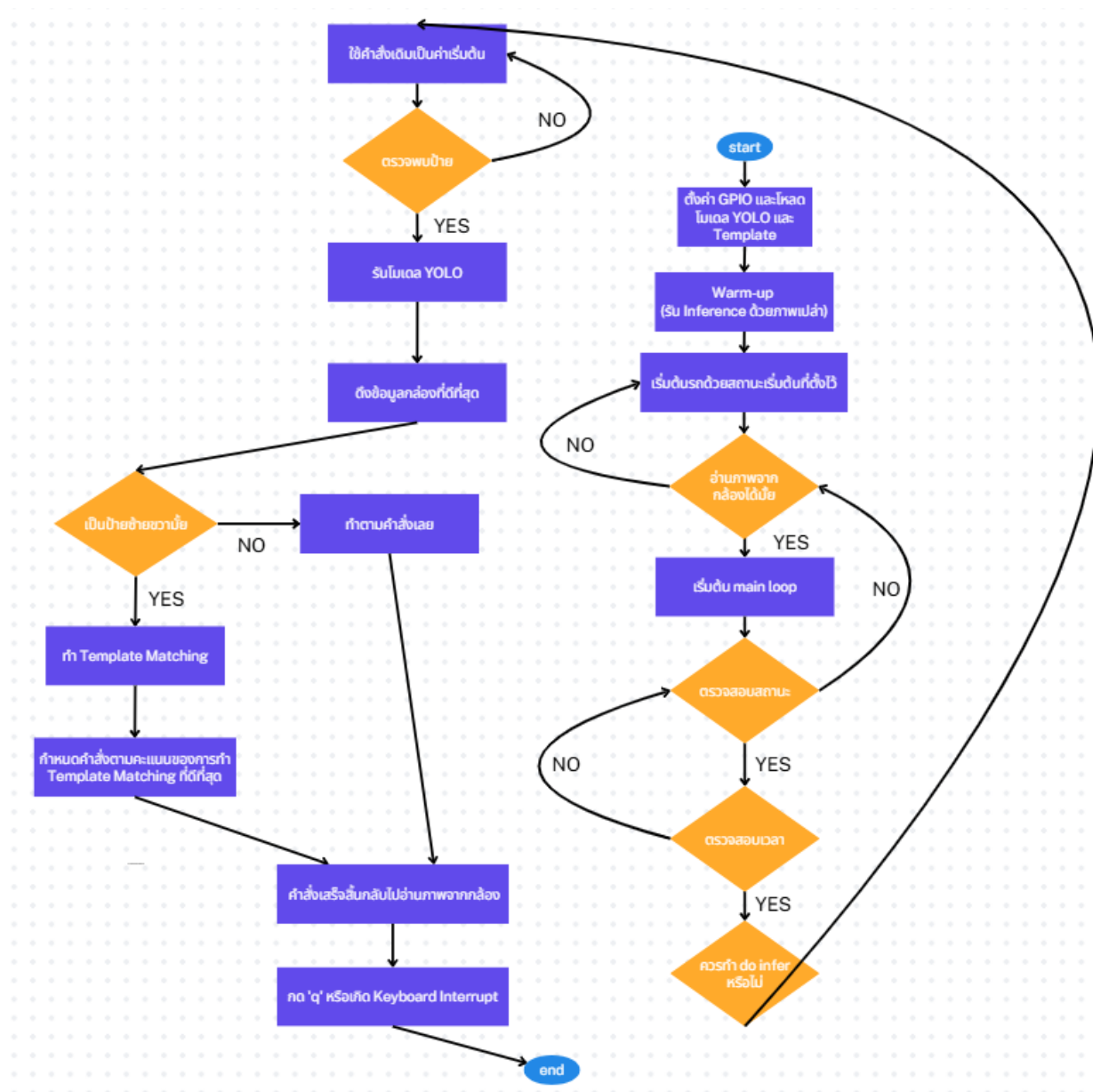
frame_i += 1
time.sleep(0.05) # รอจนเจอเล็กก่อน
```

เพื่อความแม่นยำในการตัดสินใจ คำสั่งที่ตัดสินใจได้จะถูกตรวจสอบด้วย Command Throttling โดยใช้ `COMMAND_HOLD` เพื่อป้องกันการสั่งงานมอเตอร์ซ้ำถี่เกินไป ซึ่งช่วยเพิ่มเสถียรให้กับกลไกการขับเคลื่อน คำสั่งเลี้ยว (left, right) จะมีการหน่วงเวลาด้วย `time.sleep(TURN_DUR)` เพื่อให้รถหมุนในระยะเวลาที่กำหนด ก่อนกลับมาเดินหน้าต่อ

```
except KeyboardInterrupt:
    pass
finally:
    try:
        stop(brake=True)
    except Exception:
        pass
picam2.stop()
```

และสุดท้ายเป็นการสิ้นสุดเมื่อผู้ใช้กด Ctrl+C จะเกิด KeyboardInterrupt ทำให้โปรแกรมออกจากลูปและเข้าสู่บล็อก finally เพื่อเรียกฟังก์ชันปิดระบบอย่างปลอดภัย.

3.9 Webcam flowchart



3.10 สรุปโครงสร้างของโค้ด Webcam

เริ่มต้นตั้งค่า GPIO เพื่อควบคุมมอเตอร์สองชุดโดยใช้ PWM สำหรับการปรับความเร็ว

```
import time
import atexit
import argparse, cv2, torch
from ultralytics import YOLO
import numpy as np
from skimage.metrics import structural_similarity as ssim

# --- 0. ค่าคงที่และ Template Matching
MODEL = "best.pt"
CONF = 0.20 # ใช้ค่าเริ่มต้นที่ต่ำลงตาม args
IMG_SZ = 480 # ใช้ค่าเริ่มต้นตาม args
TPL_R = "1.png" # ต้องมีไฟล์ภาพป้ายขวา
TPL_L = "2.png" # ต้องมีไฟล์ภาพป้ายซ้าย
PAD = 0.03
COMPARE_SZ = (160,160)
USE_ORB = False
W_SSIM, W_EDGE, W_ORB = 0.56, 0.44, 0.0
ALLOWED = {s.upper() for s in ("left", "right", "stop", "go")}

# --- 1. การตั้งค่า GPIO และมอเตอร์ ---
# (ใช้ PIN BCM เดิม: 17, 27, 22, 23, 18, 19)
IN1, IN2 = 17, 27
IN3, IN4 = 22, 23
ENA, ENB = 18, 19
PWM_FREQ = 1000

A_INVERT = False
B_INVERT = False

pwmA = None
pwmB = None
```

กำหนดค่าสำหรับควบคุมพฤติกรรมของรถและการจัดการสถานการณ์เคลื่อนที่ ทำการกำหนดระยะเวลาของแต่ละ Action เขียนฟังก์ชัน `_clamp_speed` เพื่อป้องกันไม่ให้มอเตอร์รับค่า Duty Cycle ที่เป็นลบหรือเกิน

```
# ปรับพารามิเตอร์
SPEED_FWD = 60
SPEED_TURN = 60
TURN_DUR = 0.5
GO_DUR = 1.0
COMMAND_HOLD = 0.0

ACTION_DURATIONS = {
    "LEFT": TURN_DUR,
    "RIGHT": TURN_DUR,
    "GO": GO_DUR,
    "STOP": 0.0,
}

# --- State Variables ---
current_action = None
action_in_progress = False
action_end_time = 0.0

# --- Motor Functions
def _clamp_speed(p):
    return max(0, min(100, int(p)))
```

กำหนดค่าควบคุมมอเตอร์ทั้งหมดให้เป็นค่า Output เพื่อให้โปรแกรมสามารถส่งสัญญาณควบคุมได้ทันที กำหนดค่าเริ่มต้นสัญญาณ PWM และมีฟังก์ชันปิดระบบอย่างปลอดภัยจะถูกเรียกใช้เสมอ เพื่อคืนค่าการควบคุมขา GPIO ให้กับระบบ

```
def setup():
    global pwmA, pwmB
    GPIO.setmode(GPIO.BCM)
    GPIO.setup([IN1, IN2, IN3, IN4, ENA, ENB], GPIO.OUT)
    GPIO.output([IN1, IN2, IN3, IN4], GPIO.LOW)
    pwmA = GPIO.PWM(ENA, PWM_FREQ)
    pwmB = GPIO.PWM(ENB, PWM_FREQ)
    pwmA.start(0)
    pwmB.start(0)
    atexit.register(_safe_shutdown)
```

เขียนฟังก์ชันเพื่อให้แน่ใจว่าตัวแปร PWM ได้รับการเริ่มต้นอย่างถูกต้องแล้ว หากยังเป็น None ระบบจะหยุดการทำงานทันที เพื่อป้องกันไม่ให้โค้ดพยายามสั่งงาน PWM ที่ยังไม่ถูกตั้งค่า เขียนฟังก์ชัน `_drive_motor_a` และ `_drive_motor_b` ควบคุมทิศทางและความเร็วเพื่อส่งสัญญาณควบคุมไปยังมอเตอร์แต่ละตัวโดยตรง

```
def _ensure_pwm():
    if pwmA is None or pwmB is None:
        raise RuntimeError("PWM not initialized: call setup() before driving motors")

def _drive_motor_a(direction: int, speed: int):
    _ensure_pwm()
    s = _clamp_speed(speed)
    d = direction
    if A_INVERT: d = -d
    if d > 0: GPIO.output(IN1, GPIO.HIGH); GPIO.output(IN2, GPIO.LOW)
    elif d < 0: GPIO.output(IN1, GPIO.LOW); GPIO.output(IN2, GPIO.HIGH)
    else: GPIO.output(IN1, GPIO.LOW); GPIO.output(IN2, GPIO.LOW)
    pwmA.ChangeDutyCycle(s if d != 0 else 0)

def _drive_motor_b(direction: int, speed: int):
    _ensure_pwm()
    s = _clamp_speed(speed)
    d = direction
    if B_INVERT: d = -d
    if d > 0: GPIO.output(IN3, GPIO.HIGH); GPIO.output(IN4, GPIO.LOW)
    elif d < 0: GPIO.output(IN3, GPIO.LOW); GPIO.output(IN4, GPIO.HIGH)
    else: GPIO.output(IN3, GPIO.LOW); GPIO.output(IN4, GPIO.LOW)
    pwmB.ChangeDutyCycle(s if d != 0 else 0)
```

เขียนฟังก์ชันย่อยเดินหน้า เลี้ยวซ้าย เลี้ยวขวา และหยุด

```
def forward(speed=SPEED_FWD):
    _drive_motor_a(-1, speed)
    _drive_motor_b(1, speed)

def backward(speed=SPEED_FWD):
    _drive_motor_a(1, speed)
    _drive_motor_b(-1, speed)

def left(speed=SPEED_TURN):
    _drive_motor_a(1, speed)
    _drive_motor_b(1, speed)

def right(speed=SPEED_TURN):
    _drive_motor_a(-1, speed)
    _drive_motor_b(-1, speed)

def stop(brake=False):
```

เขียนฟังก์ชันปิดเพื่อปิดการทำงานอย่างปลอดภัย โดยมักจะถูกเรียกใช้โดยอัตโนมัติเมื่อโปรแกรมจบการทำงาน

```
def _safe_shutdown():
    global pwmA, pwmB
    try:
        if pwmA: pwmA.stop()
    except Exception: pass
    try:
        if pwmB: pwmB.stop()
    except Exception: pass
    time.sleep(0.05)
    try:
        GPIO.cleanup()
    except Exception: pass
```

เทคนิคการเลี้ยงซ้ายขวา Template Matching เตรียมรูปภาพแม่แบบที่จะเอามาใช้คำนวณ และใช้เมตริกวัดความคล้าย ssim_score edge_corr และ orb_score นำคะแนนย่อยทั้งสามมาคำนวณเพื่อหาคะแนนรวม $combined = W_SSIM*s1 + W_EDGE*s2 + W_ORB*s3$

```
def gray(img): return cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
def edges(g): return cv2.Canny(g, 50, 150)

def ssim_score(a,b):
    if a.shape != b.shape: b = cv2.resize(b, (a.shape[1], a.shape[0]))
    s,_ = ssim(a,b,full=True)
    return float((s+1)/2)

def edge_corr(a,b):
    ae, be = edges(a), edges(b)
    if ae.shape != be.shape: be = cv2.resize(be, (ae.shape[1], ae.shape[0]))
    r = cv2.matchTemplate(ae, be, cv2.TM_CCOEFF_NORMED)
    return float(((r.max() if r.size else 0)+1)/2)

def orb_score(a,b,n=200):
    return 0.0

def combined_fast(crop, tpl_gray_small, tpl_edge_small, tpl_orb_small=None):
    cg = cv2.resize(gray(crop), COMPARE_SZ, interpolation=cv2.INTER_AREA)
    s1 = ssim_score(cg, tpl_gray_small)
    s2 = edge_corr(cg, tpl_edge_small)
    s3 = orb_score(cg, tpl_orb_small) if USE_ORB and tpl_orb_small is not None else 0.0
    combined = W_SSIM*s1 + W_EDGE*s2 + W_ORB*s3
    return combined, (s1, s2, s3)
```

ต่อไปเตรียมแม่แบบ โหลดรูปภาพซ้ายและขวา `tpl_r = cv2.imread(TPL_R)`

`tpl_l = cv2.imread(TPL_L)` จากนั้นจะทำการตรวจสอบว่าไฟล์แม่แบบทั้งสองถูกโหลดมาได้สำเร็จหรือไม่ หากไม่เจอหรือเจอไม่ครบจะทำการสร้างภาพสีดำขึ้นมาแทน การทำเช่นนี้ทำให้โปรแกรมยังคงทำงานต่อไปได้ แทนที่จะใช้การเปรียบเทียบรูปภาพ หลังจากโหลดรูปภาพแม่แบบแล้ว โค้ดจะทำการปรับปรุงและเตรียมแม่แบบให้พร้อมสำหรับการคำนวณ

```
try:
    tpl_r = cv2.imread(TPL_R)
    tpl_l = cv2.imread(TPL_L)
    if tpl_r is None or tpl_l is None:
        print(f"WARNING: Template files {TPL_R}/{TPL_L} not found. LEFT/RIGHT detection will use YOLO confidence only.")
        tpl_r = np.zeros((COMPARE_SZ[1], COMPARE_SZ[0], 3), dtype=np.uint8)
        tpl_l = np.zeros((COMPARE_SZ[1], COMPARE_SZ[0], 3), dtype=np.uint8)

    tpl_r_small_gray = cv2.resize(gray(tpl_r), COMPARE_SZ, interpolation=cv2.INTER_AREA)
    tpl_l_small_gray = cv2.resize(gray(tpl_l), COMPARE_SZ, interpolation=cv2.INTER_AREA)
    tpl_r_small_edge = edges(tpl_r_small_gray)
    tpl_l_small_edge = edges(tpl_l_small_gray)

    tpl_r_orb = tpl_l_orb = None
except Exception as e:
    print(f"Error preparing templates: {e}")
    tpl_r_small_gray = tpl_l_small_gray = np.zeros(COMPARE_SZ, dtype=np.uint8)
    tpl_r_small_edge = tpl_l_small_edge = np.zeros(COMPARE_SZ, dtype=np.uint8)
    tpl_r_orb = tpl_l_orb = None
```


เริ่มต้นการทำงานที่กำหนดโดยรับพารามิเตอร์ action เข้ามา โดยฟังก์ชันนี้จะควบคุมการเคลื่อนที่ตามคำสั่งที่ได้รับ พร้อมทั้งกำหนดเวลาสิ้นสุดหากเป็นการทำงานที่มีระยะเวลากำหนดไว้และมีฟังก์ชันหยุดการทำงานที่กำลังดำเนินอยู่ เมื่อระยะเวลาที่กำหนดไว้สำหรับการทำงานนั้นๆ ได้สิ้นสุดลงแล้ว โดยจะทำการหยุดมอเตอร์และเคลียร์สถานะการทำงานปัจจุบัน

```
def start_action(action):
    """Start action and set action_in_progress and action_end_time."""
    global action_in_progress, action_end_time, current_action
    action = action.upper()
    dur = ACTION_DURATIONS.get(action, 0.0)
    print(f"[{time.strftime('%H:%M:%S')}] START ACTION -> {action} (duration {dur:.2f}s)")
    if action == "STOP":
        stop(brake=True)
    elif action == "GO":
        forward(SPEED_FWD)
    elif action == "LEFT":
        left(SPEED_TURN)
    elif action == "RIGHT":
        right(SPEED_TURN)
    else:
        stop(brake=True)

    current_action = action
    if dur > 0:
        action_in_progress = True
        action_end_time = time.time() + dur
    else:
        # Action GO: considered immediate, but motors keep running forward
        action_in_progress = False
        action_end_time = 0.0

def finish_action():
    """Called when action duration finished: stop motors and clear state."""
    global action_in_progress, action_end_time, current_action
    print(f"[{time.strftime('%H:%M:%S')}] ACTION FINISHED -> {current_action}")
    if current_action in ("LEFT", "RIGHT"):
        start_action("STOP")
    else:
        stop(brake=True)
        current_action = None
        action_in_progress = False
        action_end_time = 0.0
```

กำหนด parse_args() ซึ่งช่วยให้โปรแกรมรับค่าการตั้งค่าต่างๆ จากบรรทัดคำสั่งได้อย่างง่ายดายและเป็นระเบียบ มีการรับค่าโมเดล YOLO ขนาดภาพ ค่าความเชื่อมั่น config อัตราเฟรม FPS จำนวนเฟรม

```
def parse_args():
    ap = argparse.ArgumentParser()
    ap.add_argument("--model", type=str, default=MODEL) # ใช้ MODEL เป็นค่าเริ่มต้น
    ap.add_argument("--imgsz", type=int, default=480)
    ap.add_argument("--conf", type=float, default=0.2)
    ap.add_argument("--width", type=int, default=640)
    ap.add_argument("--height", type=int, default=480)
    ap.add_argument("--fps", type=int, default=30)
    ap.add_argument("--skip", type=int, default=0)
    ap.add_argument("--device", type=str, default="cpu")
    ap.add_argument("--show", action="store_true", help="Display webcam feed with detection boxes.")
    return ap.parse_args()

DEFAULT_NAMES = {0:"stop", 1:"left", 2:"right", 3:"go"}
```

เข้าสู่ฟังก์ชันหลัก main() ซึ่งทำหน้าที่เป็นจุดเริ่มต้นในการตั้งค่าการทำงานทั้งหมดของโปรแกรม ตั้งค่า parse_args() ตั้งค่า device เพื่อให้ cpu ประมวลผล ตั้งค่ากล้อง Warm-up model ช่วยให้การประมวลผลเฟรมแรกๆ ที่มาจากกล้องทำได้เร็วขึ้น เริ่มต้นรถเมื่อเปิดด้วยสถานะเริ่มต้น

```
def main():
    global action_in_progress, action_end_time, current_action
    args = parse_args()
    # ... (YOLO model & Camera setup) ...
    model = YOLO(args.model)
    device = args.device
    if device != "cpu" and not torch.cuda.is_available():
        device = "cpu"
    model.to(device)
    class_names = model.names if model.names is not None else DEFAULT_NAMES

    CAMERA_INDEX = 0
    cap = cv2.VideoCapture(CAMERA_INDEX)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, args.width)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, args.height)
    cap.set(cv2.CAP_PROP_FPS, args.fps)
    if not cap.isOpened():
        print(f"Error: Could not open webcam at index {CAMERA_INDEX}")
        _safe_shutdown(); exit()
    time.sleep(0.2)
    _ = model(np.zeros((args.height, args.width, 3), dtype=np.uint8), imgsz=args.imgsz, conf=args.conf, verbose=False)

    # ----- แก๊ว: เริ่มต้น State -----
    start_action("STOP")
    print("Initial command: STOP")
    # -----

    frame_i = 0
```

เข้าสู่ฟังก์ชันหลักไปแล้วก็ต่อยด้วยลูปหลักของโปรแกรม ซึ่งทำงานซ้ำไปเรื่อยๆ

การจับภาพจากกล้องและการจัดการเฟรม

```
while True:
    ret, bgr = cap.read()
    if not ret:
        time.sleep(0.01)
        continue
    now = time.time()
```

จัดการกับสถานะที่กำลังดำเนินการอยู่ if action_in_progress ตรวจสอบว่ามีคำสั่งใดๆ ที่กำลังดำเนินการอยู่หรือไม่ ถ้า action_end_time เป็นการบอกว่าการกระทำนั้นสิ้นสุดแล้วเปลี่ยนสถานะเป็นว่าง finish_action() แต่ถ้าสถานะยังไม่สิ้นสุดจะแสดงสถานะ current_action และที่สำคัญ continue คือข้ามการตรวจจับ ในเฟรมปัจจุบันไปทันทีและเริ่มลูปใหม่ เพื่อให้การคำสั่งที่กำลังทำอยู่ดำเนินไปจนจบเวลาที่กำหนด โดยไม่ต้องเสียเวลาประมวลผลโมเดลซ้ำ

```

if action_in_progress:
    if now >= action_end_time:
        finish_action()
    else:
        ช่วงทำ Action (เช่น เลี้ยวอยู่) -> ไม่ต้องรัน Detection ซ้ำ
        if args.show:
            cv2.putText(bgr, f"CMD: {current_action} (HOLD {action_end_time - now:.2f}s)", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.5, bgr)
            cv2.imshow('YOLO Traffic Sign Control', bgr)
            if cv2.waitKey(1) & 0xFF == ord('q'): break
        continue

```

do_infer ช่วยควบคุมความสมดุลความเร็วในการประมวลผล (FPS) กับความแม่นยำการตรวจจับได้ หาก Raspberry Pi ไม่สามารถรัน YOLO ได้เร็วพอที่จะรักษา FPS ของกล้อง การเพิ่มค่า args.skip จะช่วยไหลลื่นขึ้น แต่จะมีการอัปเดตคำสั่งใหม่ช้าลงเล็กน้อย เงื่อนไขใน do_infer เงื่อนไขที่ 1 (args.skip == 0) คือ ไม่มีการข้ามเฟรม ทำให้ do_infer รัน YOLO ทุกเฟรม เงื่อนไขที่ 2 (frame_i % (args.skip + 1) == 0) คือการข้ามเฟรม ถ้า args.skip > 0 เช่น ตั้งค่าเป็น 1 (ข้าม 1 เฟรม) ไม่รัน YOLO เป็น False และค่าของ chosen_action จะใช้ค่าเดิมที่ตั้งไว้ก่อนหน้านี้คือ current_action หรือคำสั่งที่รถกำลังทำอยู่ซึ่งหมายความว่ารถจะทำคำสั่งเดิมต่อไป โดยไม่มีการประมวลผลภาพใหม่จะรัน YOLO เมื่อ frame_i % 2 == 0 เป็น True YOLO ทำการรันเพื่อตรวจจับวัตถุใหม่และกำหนด chosen_action ใหม่ ถ้าเงื่อนไข 2 ข้อ มี True จะทำการรัน YOLO กรณีถ้าเป็นป้ายซ้ายและป้ายขวาจะมีการใช้เทคนิค Template Matching เพื่อเปรียบเทียบภาพที่ Crop ได้กับป้ายต้นแบบจริง ๆ เพื่อหาคะแนนรวมที่สูงที่สุดจาก Template Matching แทนที่ผลลัพธ์จาก YOLO โดยตรง เพื่อความแม่นยำในการเลี้ยวมากขึ้น

```

do_infer = (args.skip == 0) or (frame_i % (args.skip + 1) == 0)

chosen_action = current_action # เริ่มต้นด้วย action ที่กำลังทำอยู่
if do_infer:
    results = model(bgr, imgsz=args.imgsz, conf=args.conf, verbose=False)
    detection_status = "No object detected."
    if len(results[0].boxes) > 0:
        conf_scores = results[0].boxes.conf
        if isinstance(conf_scores, torch.Tensor):
            best_index = torch.argmax(conf_scores).item()
        else:
            best_index = np.argmax(conf_scores)

        best_box = results[0].boxes[best_index]
        cls_id = int(best_box.cls.item())
        conf = float(best_box.conf.item())
        action_name_yolo = class_names.get(cls_id, "go")

```

```

# ตัดภาพป้ายจราจร (Crop)
crop = bgr[y1:y2, x1:x2].copy()
if crop.size > 0:
# 3. คำนวณคะแนนรวม
comb_r, _ = combined_fast(crop, tpl_r_small_gray, tpl_r_small_edge, tpl_r_orb)
comb_l, _ = combined_fast(crop, tpl_l_small_gray, tpl_l_small_edge, tpl_l_orb)
# 4. เลือกผลลัพธ์จาก Template Matching
chosen_action = "right" if comb_r > comb_l else "left"

detection_status = f"ID {cls_id} (YOLO:{action_name_yolo}) Refined by TM: {chosen_action} (Conf: {conf:.2f} R:{comb_r:.2f})"
else:
chosen_action = action_name_yolo
detection_status = f"ID {cls_id} ({chosen_action}) detected (Conf: {conf:.2f})."
else:
chosen_action = action_name_yolo
detection_status = f"ID {cls_id} ({chosen_action}) detected (Conf: {conf:.2f})."

```

```

if chosen_action.upper() != current_action:
start_action(chosen_action)
print(f"[{frame_i}] AI: {detection_status} -> CMD: {chosen_action.upper()} (EXECUTE)")
else:
if chosen_action.upper() == "STOP":
stop(brake=True)
print(f"[{frame_i}] AI: {detection_status} -> CMD: {chosen_action.upper()} (CONTINUE)")

if args.show:
display_frame = results[0].plot(img=bgr.copy(), line_width=2, font_size=0.7, labels=True, conf=True)
else:
display_frame = bgr.copy()

# *** แสดงผลภาพจาก Webcam ***
if args.show:
if 'display_frame' not in locals():
display_frame = bgr.copy()

cv2.putText(display_frame, f"CMD: {current_action}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
cv2.imshow('YOLO Traffic Sign Control', display_frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
break
frame_i += 1

```

ส่วนสุดท้ายปิดระบบไม่ว่าจะเป็นการถูกขัดจังหวะหรือการทำงานของโปรแกรมเสร็จสิ้นตามกระบวนการหลัก โดยจะเน้นไปที่การปิดระบบอย่างปลอดภัย (Safe Shutdown)

```

except KeyboardInterrupt:
pass
finally:
stop(brake=True)
cap.release()
if args.show: cv2.destroyAllWindows()
print("\nProgram terminated. Safely shutting down.")

if __name__ == "__main__":
setup()
try:
main()
finally:
_safe_shutdown()

```

3.11 สรุปลำดับขั้นตอนทดสอบส่วนแสดงผลภาพของกล้องและส่วนในการใช้อัลกอริทึมตรวจจับวัตถุ

1 ลำดับทดสอบการแสดงผลของกล้องแบ่งเป็น 3 ส่วนหลักๆ

1 ส่วนการเตรียมการและเริ่มต้นระบบ คือการเตรียมพร้อมระบบ โดยเริ่มต้นจากการนำเข้าโมดูลที่จำเป็น ได้แก่ Picamera2, cv2, และ time หลังจากนั้นจะเป็นการสร้างอ็อบเจกต์ Picamera2() และกำหนดค่า config สำหรับการแสดงตัวอย่าง ผ่าน create_preview_configuration() การกำหนดค่านี้เป็นการบอกให้กล้องทราบถึงความละเอียดและรูปแบบภาพที่ต้องการ จากนั้นจึงเรียกใช้ไลบรารีเริ่มต้น เพื่อเริ่มต้นกระบวนการจับภาพของกล้อง และมีการปรับให้กล้องปรับสมดุลแสง และการรับแสงให้เหมาะสมก่อนเริ่มการทำงานจริงโดยใช้การหน่วงเวลาด้วย time.sleep(2) ให้เหมาะสมก่อนเริ่มการทำงานจริง เพื่อให้ได้ภาพเฟรมแรกที่มีคุณภาพดี

```
from picamera2 import Picamera2
import cv2, time

picam2 = Picamera2()
config = picam2.create_preview_configuration()
picam2.configure(config)

picam2.start()
time.sleep(2)
```

2 ส่วนประมวลผล (while True loop) ซึ่งทำหน้าที่รับภาพจากกล้องอย่างต่อเนื่องในอัตราเฟรมที่รวดเร็วในแต่ละรอบของลูป โดยมีคำสั่ง frame = picam2.capture_array() ทำหน้าที่ดึงภาพปัจจุบันจากเซ็นเซอร์กล้องออกมาในรูปแบบของ NumPy array ซึ่งเป็น format ที่ว่า OpenCV สามารถนำไปประมวลผลได้ทันทีและแสดงภาพโดยใช้ฟังก์ชัน cv2.imshow("PiCam2 Preview", frame) เป็นตัวเปิดหน้าต่างแสดงภาพเรียลไทม์หากกดปุ่ม 'c' โปรแกรมจะเข้าสู่การบันทึกภาพและทำให้สามารถมองภาพจากกล้องเพื่อดูว่ากล้องใช้งานได้มั้ยหรือต้องปรับแก้อะไรก็สามารถดูได้เลย หากต้องการออกกดปุ่ม 'q' ลูปจะถูกทำลายด้วยคำสั่ง break เพื่อยุติการทำงานของโปรแกรม

```
while True:
    frame = picam2.capture_array() # ????????? numpy array
    cv2.imshow("PiCam2 Preview", frame)

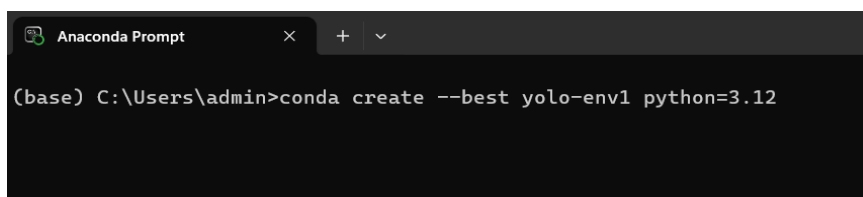
    key = cv2.waitKey(1) & 0xFF
    if key == ord('c'):
        filename = f"capture_{int(time.time())}.jpg"
        cv2.imwrite(filename, frame)
        print("?????:", filename)
    elif key == ord('q'):
        break
```

3 ส่วนสุดท้ายเป็นการล้างทรัพยากรกล้องและหน่วยความจำให้เป็นอิสระเพื่อพักการทำงานของระบบเมื่อออกจากลูป โปรแกรมจะเรียกใช้ `cv2.destroyAllWindows()` เพื่อปิดหน้าต่างแสดงภาพ OpenCV ทั้งหมดที่เปิดค้างไว้ ตามด้วย `picam2.stop()` เพื่อปิดการทำงานของกล้องอย่างเป็นระเบียบ ทำให้มั่นใจได้ว่าระบบจะถูกปิดอย่างสมบูรณ์และพร้อมสำหรับการเริ่มต้นใหม่ในครั้งต่อไป

```
cv2.destroyAllWindows()
picam2.stop()
```

2 วิธีสร้างโมเดล YOLOv8

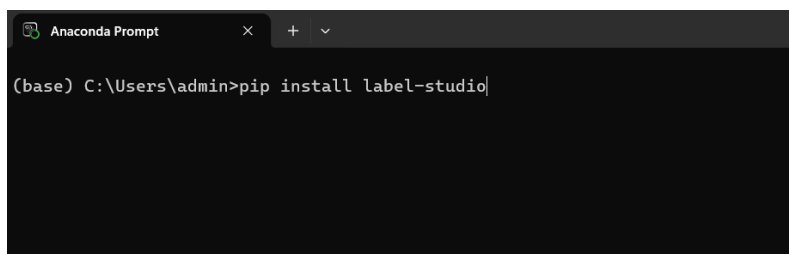
เตรียมรูปภาพและทำการตีกรอบด้วย Label studio เข้า anaconda prompt และสร้าง environment และเปิดใช้งานด้วยคำสั่ง `conda activate yolo-env1`



```

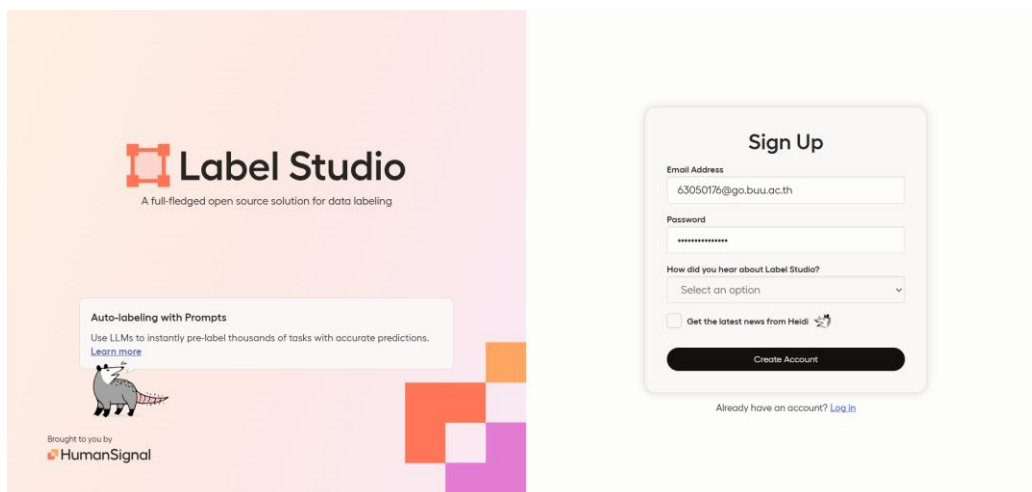
Anaconda Prompt
(base) C:\Users\admin>conda create --best yolo-env1 python=3.12
  
```

ติดตั้ง label studio ด้วยคำสั่ง และเริ่มเปิดด้วยคำสั่ง `label-studio start` เข้าหน้าเว็บกดไปที่ sign up

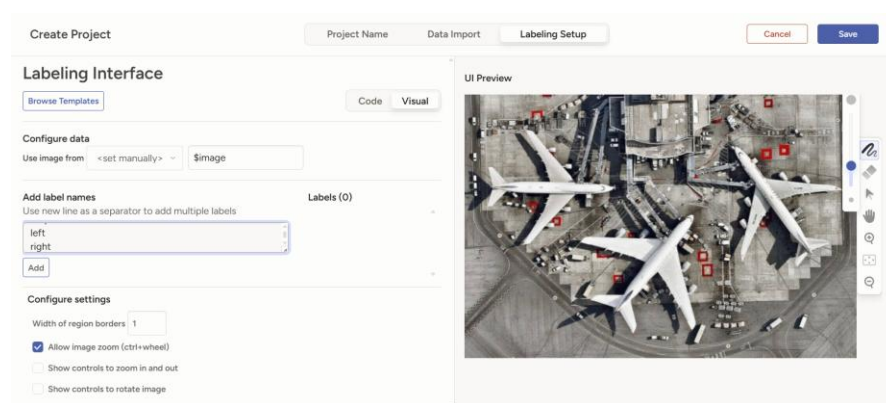
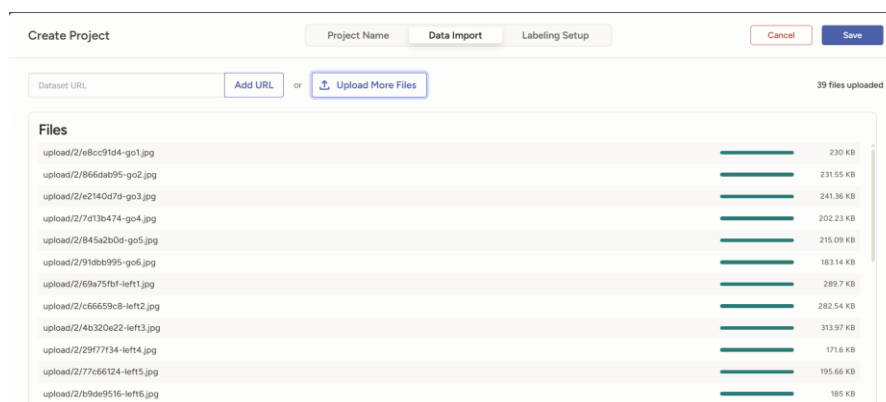


```

Anaconda Prompt
(base) C:\Users\admin>pip install label-studio
  
```



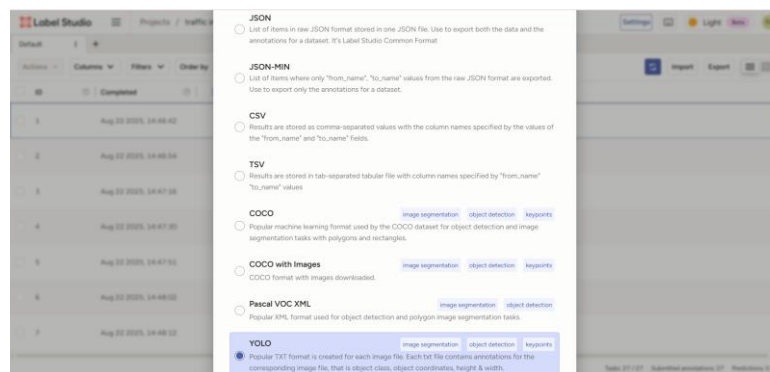
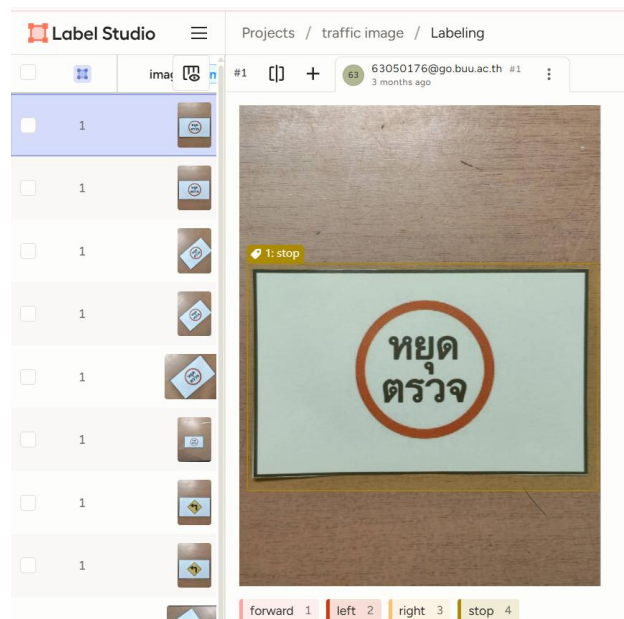
ไปที่ create project และ upload รูปภาพ เลือก label setup เปลี่ยนเป็นชื่อที่เราจะใช้ในการติกรอบ



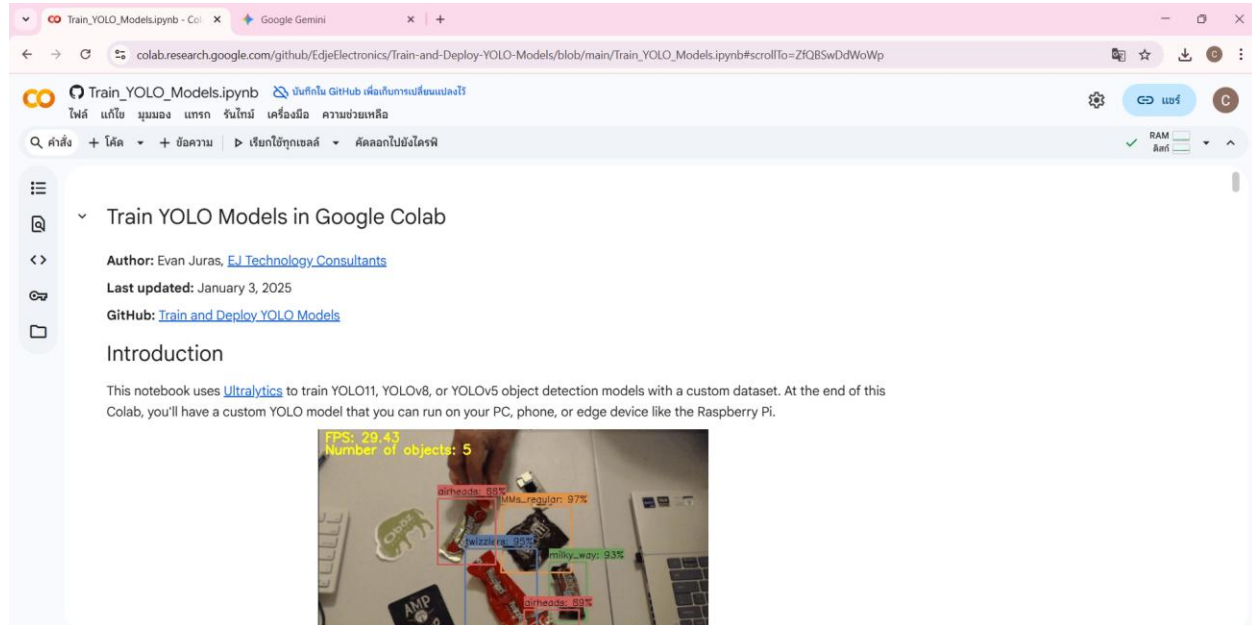
ทำการติกรอบ เมื่อติเสร็จทำการ export เลือก YOLO เพื่อนำไปสู่การ train ต่อไป



ตัวอย่างรูปที่เสร็จแล้ว



เข้า https://colab.research.google.com/github/EdgeElectronics/Train-and-Deploy-YOLO-Models/blob/main/Train_YOLO_Models.ipynb เลือก runtime และตั้งค่า เสร็จแล้วทำตามแต่ละ block



เมื่อทำครบขั้นตอนจะได้ไฟล์ดังรูป เป็นอันเสร็จสิ้น

ชื่อ	วันที่ปรับเปลี่ยน	ชนิด	ขนาด
train	22/08/2025 15:18	โฟลเดอร์แฟ้ม	
best.pt	22/08/2025 15:18	แฟ้ม PT	55,807 KB

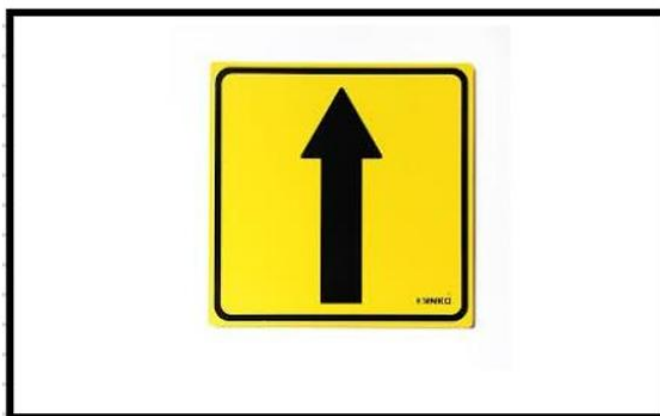
3 โค้ดตรวจสอบอัลกอริทึมที่ตรวจจับวัตถุ โดยมีรูปป้าย 4 รูปที่ใช้ในการทดสอบ



รูปเลี้ยวขวา (right)



รูปเลี้ยวซ้าย (left)



รูปเดินหน้า (forward)



รูปหยุดรถ (stop)

สรุปโค้ดการตรวจสอบอัลกอริทึมตรวจจับวัตถุ เริ่มแรกสุดเป็นการตั้งค่าและการเตรียมทรัพยากร เริ่มต้นด้วยการเตรียมองค์ประกอบที่จำเป็นก่อนเริ่มทำงาน เช่น ไลบรารี argparse, cv2, torch, YOLO, และ numpy เป็นต้น ฟังก์ชัน parse_args() เกณฑ์ความเชื่อมั่น (--conf) ขนาดภาพที่ใช้ในการประมวลผลและอุปกรณ์ที่ใช้ในการรัน

```
import argparse
import cv2
import torch
from ultralytics import YOLO
import numpy as np
import time

def parse_args():
    """ตั้งค่า argument สำหรับโมเดลและกล้อง"""
    ap = argparse.ArgumentParser(description="YOLOv8 Real-Time Detection for Testing.")
    ap.add_argument("--model", type=str, default="best.pt", help="Path to YOLO model weights.")
    ap.add_argument("--imgsz", type=int, default=480, help="Inference size (square).")
    ap.add_argument("--conf", type=float, default=0.2, help="Confidence threshold.")
    ap.add_argument("--width", type=int, default=640, help="Camera width.")
    ap.add_argument("--height", type=int, default=480, help="Camera height.")
    ap.add_argument("--fps", type=int, default=30, help="Camera FPS.")
    ap.add_argument("--skip", type=int, default=0, help="Skip N frames between inference.")
    ap.add_argument("--device", type=str, default="cpu", help="Device to use (e.g., 'cpu' or 'cuda:0').")
    return ap.parse_args()

def run_yolo_test():
    """รัน YOLO Detection บน Webcam และแสดงผล"""
    args = parse_args()
```

```
try:
    model = YOLO(args.model)
except Exception as e:
    print(f"Error loading model: {e}")
    print("Please check if 'best.pt' exists in the current directory.")
    return

device = args.device
if device != "cpu" and not torch.cuda.is_available():
    device = "cpu"
model.to(device)
```

หลังจากนั้นจะเริ่มขั้นตอนโหลดโมเดล YOLO และ Warm up model โดยเป็นการรันโมเดลด้วยภาพว่างหนึ่งครั้ง เพื่อลดค่าความหน่วง (Latency) ของการตรวจจับเฟรมแรกและลดภาระหน่วยความจำของ GPU

```
print("Warming up YOLO model...")
_ = model(np.zeros((args.height, args.width, 3), dtype=np.uint8),
          imgsiz=args.imgsz, conf=args.conf, verbose=False)
print("Ready. Press 'q' to exit.")
```

ต่อมาทำการตั้งค่ากล้องและตั้งค่าคุณสมบัติต่าง ๆ เช่น ความละเอียดและความเร็วเฟรม (FPS)

```
CAMERA_INDEX = 0 # 0 คือกล้องหลักของระบบ
cap = cv2.VideoCapture(CAMERA_INDEX)

cap.set(cv2.CAP_PROP_FRAME_WIDTH, args.width)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, args.height)
cap.set(cv2.CAP_PROP_FPS, args.fps)

if not cap.isOpened():
    print(f"Error: Could not open webcam at index {CAMERA_INDEX}")
    return

time.sleep(0.2)
```

ตั้งค่าทุกอย่างและเตรียมพร้อมทุกอย่างก็จะมาเข้าสู่ลูปการทำงาน while True ทำหน้าที่อ่านภาพจากกล้องที่ละเฟรมและรันการตรวจจับอย่างต่อเนื่องในแต่ละรอบของลูป จะมีการดึงภาพปัจจุบันจากเว็บแคมด้วยฟังก์ชัน cap.read() และจะมีการข้ามเฟรมเมื่อ skip ไม่เป็น 0 จะทำการข้ามการตรวจจับบางเฟรม เพื่อลดภาระการประมวลผลบน CPU/GPU และมีการเรียกใช้ model(bgr, ...) เพื่อทำการทำนายผลลัพธ์การตรวจจับ และสุดท้ายมีการแสดงผลของผลลัพธ์จาก Ultralytics จะถูกใช้ในการวาด Bounding Box และ Label ลงบนภาพโดยอัตโนมัติ จากนั้นภาพที่ได้รับการตรวจจับจะถูกแสดงผลบนหน้าต่าง OpenCV ด้วย cv2.imshow() พร้อมการแสดงค่า FPS เพื่อให้ทราบถึงการทำงานของระบบ

```

frame_i = 0
start_time = time.time()

try:
    while True:
        ret, bgr = cap.read() # อ่านภาพ
        if not ret:
            print("Error: Failed to capture image from webcam.")
            break

        # 3. รับ Inference ตามการตั้งค่า --skip
        do_infer = (args.skip == 0) or (frame_i % (args.skip + 1) == 0)

        if do_infer:
            # รับ YOLO prediction
            results = model(bgr, imgsz=args.imgsz, conf=args.conf, verbose=False)

            # ใช้ .plot() เพื่อวาดกล่อง Bounding Box และ Label บนภาพพื้นที่
            annotated_frame = results[0].plot()

        else:
            # ถ้า skip ไม่ใช้ภาพสุดท้ายที่มีการวาดแล้ว (ถ้ามี) หรือใช้ภาพเดิม
            if 'annotated_frame' not in locals():
                annotated_frame = bgr.copy()

        # 4. แสดงผลบนหน้าจอ
        cv2.putText(annotated_frame, f"FPS: {cap.get(cv2.CAP_PROP_FPS):.1f}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0),
        cv2.imshow("YOLO Test Display (Press Q to Quit)", annotated_frame)

        # ตรวจสอบการกด 'q' เพื่อออก
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

        frame_i += 1

```

สุดท้ายคือการทำความสะดวกสบายระบบ เพื่อป้องกันการล๊อคกล้องไม่ให้โปรแกรมอื่นใช้งานได้ต่อและหลีกเลี่ยงการใช้ทรัพยากรหน่วยความจำที่ค้างอยู่เป็นการรับรองว่าโปรแกรมจะปิดตัวลงอย่างถูกต้องทั้งหมด มีการตรวจสอบการกดปุ่ม 'q' ด้วย cv2.waitKey(1) เป็นการออกจากวงวนหลัก เมื่อวงวนสิ้นสุดลงหรือเมื่อเกิดข้อผิดพลาดบล็อก finally จะทำงานเสมอ โดยการเรียกใช้ cap.release() เพื่อปิดการทำงานของเว็บแคม และ cv2.destroyAllWindows() เพื่อปิดหน้าต่างแสดงผลที่เปิดไว้ทั้งหมด

```

except KeyboardInterrupt:
    pass
finally:
    cap.release()
    cv2.destroyAllWindows()
    print("\nProgram terminated.")

if __name__ == "__main__":
    run_yolo_test()

```

ผลลัพธ์ตัวอย่างของการทดสอบ

โค้ดทดสอบ F1 score

โค้ดเริ่มต้นด้วยการนำเข้าไลบรารีที่จำเป็น ได้แก่ argparse สำหรับการตั้งค่าพารามิเตอร์ของโปรแกรม (เช่น เส้นทางโมเดล, ขนาดภาพ, และเกณฑ์ความเชื่อมั่น), cv2 (OpenCV) สำหรับการจัดการกล้องและการแสดงผล, torch สำหรับการประมวลผลด้วย GPU , และ ultralytics สำหรับการใช้งานโมเดล YOLO โดยตรง

นอกจากนี้ยังมีการนำเข้าไลบรารี supervision ซึ่งเป็นเครื่องมือสำคัญในการจัดการผลลัพธ์การตรวจจับและคำนวณเมตริกต่าง ๆ

```
import argparse
import cv2
import torch
from ultralytics import YOLO
import numpy as np
import time
import os
import supervision as sv # นำเข้า supervision เพื่อใช้ sv.Detections และ F1Score
from supervision.metrics import F1Score

# --- ส่วนที่ 1: YOLO WebCam Inference (การตรวจจับพร้อม Argument Parser) ---

def parse_args():
    """ตั้งค่า argument สำหรับโมเดลและกล้อง"""
    ap = argparse.ArgumentParser(description="YOLOv8 Real-Time Detection and F1 Score Simulation.")
    # แก้ไข default path ให้ยืดหยุ่นขึ้น (ถ้าไม่ใช่ Hardcode) แต่ยังคง Path เดิมตามที่ผู้ใช้กำหนด
    ap.add_argument("--model", type=str, default="/home/bosss/best.pt", help="Path to YOLO model weights (e.g., yolov8n.pt or best.pt).")
    ap.add_argument("--imgsz", type=int, default=480, help="Inference size (square).")
    ap.add_argument("--conf", type=float, default=0.25, help="Confidence threshold.")
    ap.add_argument("--device", type=str, default="cpu", help="Device to use (e.g., 'cpu' or 'cuda:0').")
    return ap.parse_args()
```

ต่อมาเข้าสู่ฟังก์ชัน run_yolo_test() ขั้นตอนแรกคือการเตรียมโมเดลให้พร้อมสำหรับการทำงาน โค้ดจะเริ่มจากการดึงค่าพารามิเตอร์ที่จำเป็น เช่น เส้นทางไฟล์น้ำหนักของโมเดล, ขนาดภาพ, และเกณฑ์ความเชื่อมั่น จากนั้นจึงใช้ไลบรารี ultralytics ในการโหลดโมเดลเข้าสู่หน่วยความจำ เมื่อโมเดลพร้อมแล้ว โปรแกรมจะเปิดใช้งานเว็บแคม หากกล้องเปิดสำเร็จ โปรแกรมจะเข้าสู่ ลูปหลัก มีการจับภาพในแต่ละรอบของลูปอ่านภาพเฟรมปัจจุบันจากกล้อง และทำนายเฟรมที่อ่านได้จะถูกป้อนเข้าสู่โมเดล ซึ่งโมเดลจะทำการทำนายตำแหน่งและประเภทของวัตถุทั้งหมดที่ปรากฏในเฟรมนั้น ผลลัพธ์จากการทำนายจะถูกเปลี่ยนให้เป็นภาพที่มีการทำเครื่องหมายอย่างชัดเจน มีการแก้ไขภาพสะท้อนเนื่องจากกล้องเว็บแคมส่วนใหญ่มักแสดงภาพในลักษณะ สะท้อนเหมือนกระจก ซึ่งอาจทำให้ผู้ใช้งานสับสน จึงใช้ cv2.flip(annotated_frame, 1) เพื่อพลิกภาพในแนวนอนซึ่งเป็นการแก้ไขปัญหานี้ ทำให้ภาพที่แสดงผลตรงกับทิศทางจริง ภาพที่ผ่านการประมวลผลแล้วจะถูกแสดงบนหน้าจอด้วย cv2.imshow() และโปรแกรมจะตรวจสอบการกดแป้นพิมพ์ หากกด 'q' ลูปจะสิ้นสุดลง และโปรแกรมจะทำการปิดกล้องและหน้าต่างแสดงผลทั้งหมดอย่างถูกต้อง

```
def run_yolo_test():
    """รับ YOLO Detection บน Webcam และแสดงผล"""
    print("---- 1. เริ่ม YOLO WebCam Inference ----")

    # ตรวจสอบว่ามี Argument เข้ามาหรือไม่ (ใช้ค่า Default ถ้าไม่มี)
    try:
        args = parse_args()
    except SystemExit:
        # กรณีรับในสภาพแวดล้อมที่ไม่ได้รองรับ sys.argv (เช่น บน IDE)
        class MockArgs:
            model = "/home/bosss/best.pt"
            imgsz = 240
            conf = 0.25
            device = "cpu"
        args = MockArgs()

    # 1. โหลดโมเดล YOLO
    try:
        model = YOLO(args.model)
    except Exception as e:
        print(f"❌ Error loading model: {e}")
        print(f"โปรดตรวจสอบว่าไฟล์ '{args.model}' มีอยู่ในไดเรกทอรีปัจจุบัน หรือเส้นทางถูกต้อง")
        return

    # กำหนด Device
    device = args.device
    if device != "cpu" and not torch.cuda.is_available():
        device = "cpu"
    print(f"Running on device: {device}")
    model.to(device)

    # 2. ตั้งค่ากล้อง
    CAMERA_INDEX = 1 # 0 คือกล้องหลักของระบบ
    cap = cv2.VideoCapture(CAMERA_INDEX)
```

```
if not cap.isOpened():
    print(f"❌ Error: ไม่สามารถเปิดกล้องได้ที่ดัชนี {CAMERA_INDEX}")
    return

# Warmup YOLO
print("Warming up YOLO model...")
_ = model(np.zeros((args.imgsz, args.imgsz, 3), dtype=np.uint8),
          imgsz=args.imgsz, conf=args.conf, verbose=False)
print("✅ Ready. Press 'q' to exit Webcam Detection.")

try:
    while True:
        ret, bgr = cap.read() # อ่านภาพ
        if not ret:
            print("Error: Failed to capture image from webcam.")
            break

        # รับ YOLO prediction
        results = model(bgr, imgsz=args.imgsz, conf=args.conf, verbose=False)

        # ใช้ .plot() เพื่อวาดกล่อง Bounding Box และ Label บนภาพทันที
        annotated_frame = results[0].plot()

        # 🪞 เพิ่มการพลิกภาพเพื่อปิด Mirror Effect (ตามที่ได้แก้ไขไว้ก่อนหน้านี้)
        annotated_frame = cv2.flip(annotated_frame, 1)

        # 4. แสดงผลบนหน้าจอ
        cv2.imshow('YOLO Real-Time Detection', annotated_frame)

        # ตรวจสอบการกด 'q' เพื่อออก
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
```

```
except KeyboardInterrupt:
    print("\nProgram interrupted.")
finally:
    cap.release()
    cv2.destroyAllWindows()
    print("--- สิ้นสุด YOLO WebCam Inference ---")
```

การจำลองและการประเมินประสิทธิภาพด้วย F1 Score ของโค้ดคือฟังก์ชัน `simulate_f1_score()` เป็นการจำลองสถานการณ์การประเมินผลโมเดล เริ่มต้นด้วยการสร้างเครื่องมือประเมินผลโดยใช้คลาส `F1Score` จากไลบรารี `Supervision` ช่วยจัดการการคำนวณเมตริกสำหรับงาน Computer Vision ได้อย่างมีประสิทธิภาพ ในการประเมินนี้ ได้กำหนดให้ประเมินเฉพาะวัตถุที่มี Class ID 0 และ 1 เป็นหลัก และตั้งค่า เกณฑ์ IoU ไว้ที่ 0.5 เกณฑ์นี้ทำหน้าที่เป็นตัวตัดสินว่า Bounding Box ที่โมเดลทำนายนั้น "ถูกต้อง" หรือไม่เมื่อเทียบกับ Ground Truth (ข้อมูลจริง) หากพื้นที่ทับซ้อนเกินกว่า 50% ของพื้นที่รวมทั้งหมด จึงจะถือว่าเป็น True Positive จากนั้นได้สร้างชุดข้อมูลจำลองขึ้นมา 2 ภาพ โดยใช้โครงสร้างข้อมูล `sv.Detections` เพื่อให้เห็นภาพความแตกต่างระหว่างผลลัพธ์ในอุดมคติและผลลัพธ์ที่มีข้อผิดพลาด ส่วน Prediction คือผลลัพธ์ที่สมมติว่าโมเดลทำนายได้ ชุดข้อมูลจำลองนี้แสดงให้เห็นสถานการณ์ต่าง ๆ เช่น ภาพที่ 1 ซึ่งทำนายได้ใกล้เคียงกับความจริงและภาพที่ 2 ซึ่งมีข้อบกพร่อง เช่น การทำนายเกินจริงหรือการตรวจจับพลาด เมื่อชุดข้อมูลจำลองถูกป้อนเข้าสู่เครื่องมือ F1 Score จะทำการสะสมจำนวน True Positive, False Positive, และ False Negative จากทุกภาพ เมื่อสิ้นสุดการจำลอง `result()` จะถูกเรียกใช้เพื่อคำนวณค่าสุดท้ายของเมตริกสำคัญ เช่น Precision (ความแม่นยำ) Recall (ความสมบูรณ์) และ F1 Score โดยมีสูตรคือ $F1\ Score = 2 \times (ความแม่นยำ \times ความสมบูรณ์ / ความแม่นยำ + ความสมบูรณ์)$

```
def simulate_f1_score():
    """
    ฟังก์ชันสำหรับจำลองการคำนวณ F1 Score บนชุดข้อมูลสมมติ (ใช้สำหรับการประเมินโมเดล)
    """
    print("\n--- 2. เริ่มจำลองการคำนวณ F1 Score (Validation) ---")

    # กำหนดคลาสที่ใช้ในการประเมิน (0=person, 1=bicycle ในข้อมูลจำลองนี้)
    class_ids_to_evaluate = [0, 1]

    # สร้าง F1Score calculator ที่ IoU 0.50
    f1_calculator = F1Score(
        class_ids=class_ids_to_evaluate,
        iou_threshold=0.5
    )

    # --- ข้อมูลจำลองสำหรับ 2 ภาพ (Ground Truth vs. Prediction) ---
    # **ภาพที่ 1: Perfect Match
    targets_1 = sv.Detections(
        xyxy=np.array([[10, 10, 100, 100], [200, 200, 300, 300]]),
        class_id=np.array([0, 1])
    )
    detections_1 = sv.Detections(
        xyxy=np.array([[15, 15, 105, 105], [205, 205, 305, 305]]),
        class_id=np.array([0, 1]),
        confidence=np.array([0.9, 0.8])
    )

    # **ภาพที่ 2: มี False Positive (FP) และ False Negative (FN)
    targets_2 = sv.Detections(
        xyxy=np.array([[10, 10, 100, 100], [400, 400, 500, 500]]), # GT มี 2 คน
        class_id=np.array([0, 0])
    )
    detections_2 = sv.Detections(
        xyxy=np.array([[15, 15, 105, 105], [600, 600, 700, 700]]), # ทำนายถูก 1 คน และทำนายผิดเป็นจักรยาน 1 คัน (FP)
        class_id=np.array([0, 1]),
        confidence=np.array([0.9, 0.7])
    )
```


บทที่ 4

ทดสอบโครงการ

4.1 ตาราง Unit test

แบบที่	แบบทดสอบ	ทดสอบครั้งที่ 1- 10 สำเร็จ	ทดสอบครั้งที่ 11- 20 สำเร็จ	ทดสอบครั้งที่ 21- 30 สำเร็จ
1	สั่งเดินหน้า	8	9	9
2	สั่งเลี้ยวซ้าย	9	7	8
3	สั่งเลี้ยวขวา	6	10	8
4	สั่งหยุด	8	8	10
5	เจอป้ายเดินหน้า	9	8	10
6	เจอป้ายเลี้ยวซ้าย	7	4	6
7	เจอป้ายเลี้ยวขวา	6	10	8
8	เจอป้ายหยุด	10	10	10

สรุปผลการทดสอบของ unit test

- 1 การทดสอบแบบที่ 1 มีประสิทธิภาพโดยรวมที่ดี โดยบางช่วงตรวจจับป้ายผิดเป็นป้ายหยุด เกิดจากสภาพแวดล้อมในการทดสอบที่ไม่ดี กล้องไปจับภาพอื่นแล้วแปลงสัญญาณผิด
- 2 การทดสอบแบบที่ 2 โดยในระยะแรกมีการทำงานที่ดีมีประสิทธิภาพแต่พอผ่านมาสักระยะเกิดการแกว่ง ทำให้มีปัญหาห้วงมอเตอร์ซ้ายและขวาในการเลี้ยวซ้าย อาจจะมาจากสมดุลการเลี้ยวที่บิดพลาดไป และอัลกอริทึมประมวลผลภาพผิดระหว่างซ้ายกับขวาจึงทำให้เกิดข้อผิดพลาดขึ้น
- 3 การทดสอบแบบที่ 3 ในผลการทดสอบไม่ดีเลยเกิดจากการปรับมอเตอร์ผิดพลาดไป ใส่ time.sleep น้อยหรือมากเกินไปการคุมรถจึงไม่เสถียร และอัลกอริทึมประมวลผลภาพผิดระหว่างซ้ายกับขวาจึงทำให้เกิดข้อผิดพลาดขึ้น

4 การทดสอบแบบที่ 4 ในช่วงแรกรถมองป้ายหยุดเป็นป้ายเดินหน้าหรืออัลกอริทึมตรวจจับไม่เจอจึงมีผลลาดอยู่ข้างแต่ในช่วงท้าย ทำการปรับตำแหน่งภาพทำให้รถทำการหยุดได้ 100%

5 การทดสอบแบบที่ 5 เจอป้ายเดินหน้าได้เกือบ 100% แต่มีบางช่วงกล้องหลุดโฟกัสทำให้เกิดข้อผิดพลาด

6 การทดสอบแบบที่ 6 มีข้อผิดพลาดกล้องกลับกันทำให้จากตรวจจับเลี้ยวซ้ายเป็นการตรวจจับเลี้ยวขวา

7 การทดสอบแบบที่ 7 มีข้อผิดพลาดเหมือนการทดสอบแบบที่ 6 คือกล้องกลับกันทำให้จากตรวจจับเลี้ยวขวาเป็นการตรวจจับเลี้ยวซ้าย

8 การทดสอบแบบที่ 8 เจอป้ายหยุดนั้นได้ 100% มาจากที่ป้ายหยุดต่างกับป้ายอันอื่นมากที่สุดการทำงานจึงสมบูรณ์ถ้าอยู่ในระยะที่กล้องจับเฟรมได้

4.2 ตาราง Systems test

แบบที่	แบบทดสอบ	ทดสอบครั้งที่ 1- 10 สำเร็จ	ทดสอบครั้งที่ 11- 20 สำเร็จ	ทดสอบครั้งที่ 21- 30 สำเร็จ
1	ไปข้างหน้าแบบไม่มีป้าย	10	10	10
2	ไปข้างหน้ามีป้ายตรงไป	10	10	10
3	เจอป้ายเลี้ยวขวา	4	7	6
4	เจอป้ายเลี้ยวซ้าย	7	8	5
5	เจอป้ายหยุด	10	10	10

สรุปผลการทดสอบของ systems test

1 การทดสอบแบบที่ 1 มีประสิทธิภาพโดยรวมที่ดีเนื่องจากไม่ได้ใช้อัลกอริทึมในการประมวลผล ใช้แค่ฟังก์ชันในบอร์ดอย่างเดียวไม่มีการทำงานร่วมกันทำให้การทำงานมีความถูกต้องไม่มีข้อผิดพลาด

2 การทดสอบแบบที่ 2 มีประสิทธิภาพโดยรวมที่ดี ป้ายมีลักษณะต่างกับป้ายอื่น และถึงกล้องจะประมวลผลกลับป้ายก็ยังเป็นลักษณะเดิมไม่ไปชนกับป้ายอื่นทำให้การทำงานไม่มีความผิดพลาด ถ้าอยู่ในระยะที่กล้องจับป้ายได้ไม่หลุดโฟกัสของกล้อง

3 การทดสอบแบบที่ 3 มีผลการทดสอบที่ไม่ดีต้องทำการแก้ไข เนื่องมาจากการประมวลผลที่พลาดไปสลับกับป้าย เลี้ยวซ้ายทำให้รถเลี้ยวผิดพลาด อาจต้องทำการเทรน YOLO ใหม่หรือปรับเฟรมเรตกล้อง ลดขนาดกล้องให้รับ ภาพลดลง เพื่อให้การประมวลผลนั้นดีขึ้น ใช้ฟังก์ชันซ้อนในการประมวลผลภาพป้ายเลี้ยวซ้ายและป้ายเลี้ยวขวา เพื่อให้เกิดความแม่นยำให้ไม่มีข้อผิดพลาด

4 การทดสอบแบบที่ 4 ลักษณะเดียวกับผลการทดสอบแบบที่ 3 มีผลการทดสอบที่ไม่ดีต้องทำการแก้ไข เนื่องมาจากการประมวลผลที่พลาดไปสลับกับป้ายเลี้ยวซ้ายทำให้รถเลี้ยวผิดพลาด อาจต้องทำการเทรน YOLO ใหม่หรือปรับเฟรมเรตกล้อง ลดขนาดกล้องให้รับภาพลดลง เพื่อให้การประมวลผลนั้นดีขึ้น ใช้ฟังก์ชันซ้อนในการ ประมวลผลภาพป้ายเลี้ยวซ้ายและป้ายเลี้ยวขวาเพื่อให้เกิดความแม่นยำให้ไม่มีข้อผิดพลาด

5 การทดสอบแบบที่ 5 มีประสิทธิภาพโดยรวมที่ดีเหมือนการทดสอบแบบที่ 2 ป้ายมีลักษณะต่างกับป้ายอื่น และ ถึงกล้องจะประมวลผลกลับป้ายก็ยังเป็นลักษณะเดิมไม่ไปชนกับป้ายอื่นทำให้การทำงานไม่มีความผิดพลาด ถ้าอยู่ในระยะที่กล้องจับป้ายได้ไม่หลุดโฟกัสของกล้อง

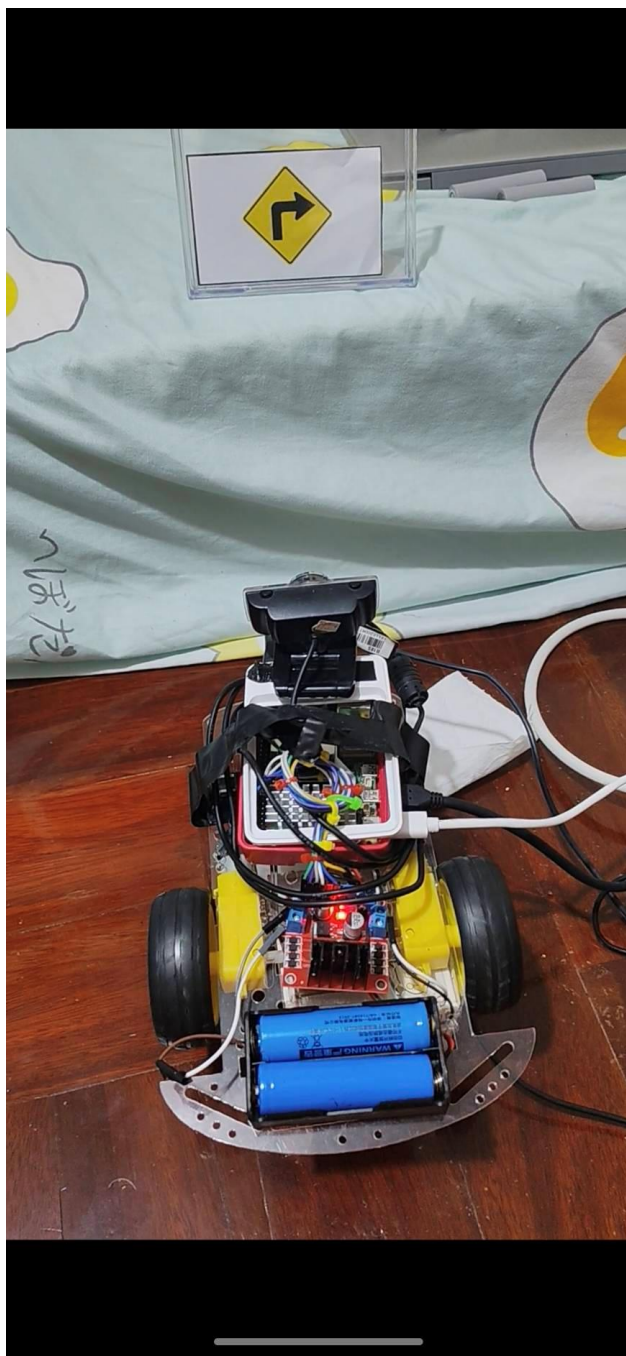
4.3 สรุปการทดสอบของโครงการ

ผลการทดสอบแสดงให้เห็นว่าระบบมีแนวโน้มที่จะทำงานได้ดีในสถานการณ์เจอป้ายเดินทางหน้าและเจอ ป้ายหยุดอย่างชัดเจน แต่ถ้าเจอสถานการณ์ที่เป็นการเจอป้ายเลี้ยวซ้ายและขวาอาจจะมีผลผิดพลาดอยู่บ้าง ซึ่งเป็นผลเกี่ยวกับการประมวลผลของกล้อง เช่น ความหน่วง การประมวลผล การ stream ของกล้องและตัว YOLO ที่อาจจะมีภาระที่มากเกินไปทำให้เกิดข้อผิดพลาดบ้าง ซึ่งถ้าไม่เอาป้ายมาเกี่ยวข้องการทำงานรถถือว่า เป็นไปได้ด้วยดี ความผิดพลาดน้อย และการที่ป้ายเลี้ยวมีลักษณะที่คล้ายกันทำให้ข้อผิดพลาดตรงนี้มีอยู่บ้าง

4.4 รูปการทดสอบ



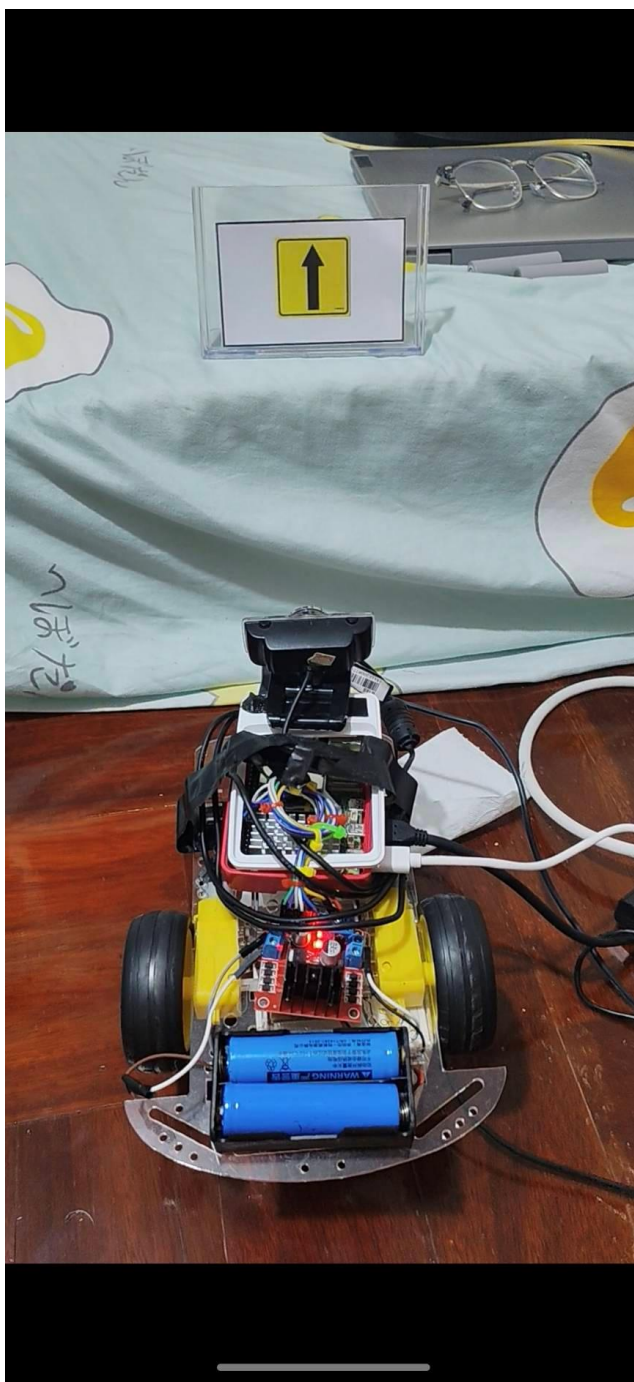
รูปการทดสอบเมื่อกล้องเจอป้ายหยุด ให้รถหยุดตามป้าย



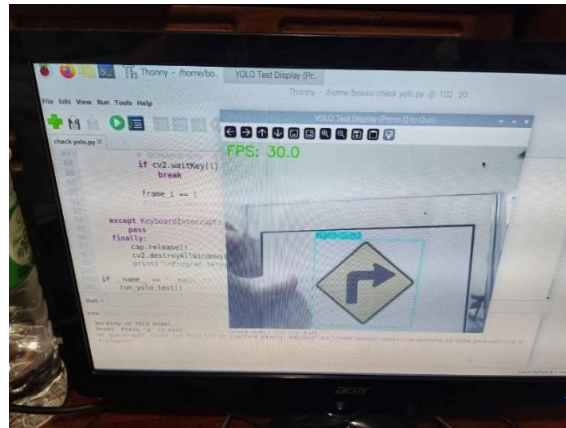
รูปการทดสอบเมื่อกล้องเจอป้ายเลี้ยวขวา ให้รถทำการเลี้ยวขวา



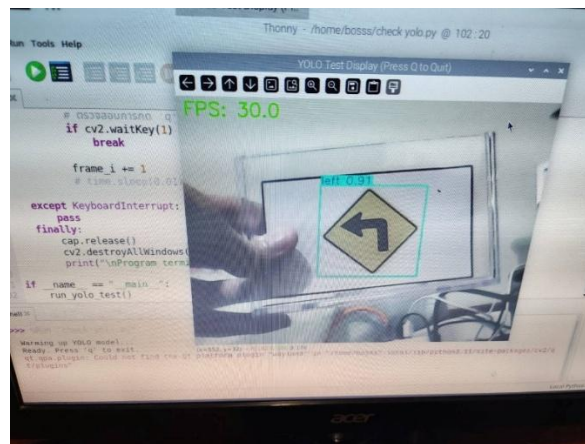
รูปการทดสอบเมื่อกล้องเจอบ้ายเลี้ยวซ้าย ให้รถทำการเลี้ยวซ้าย



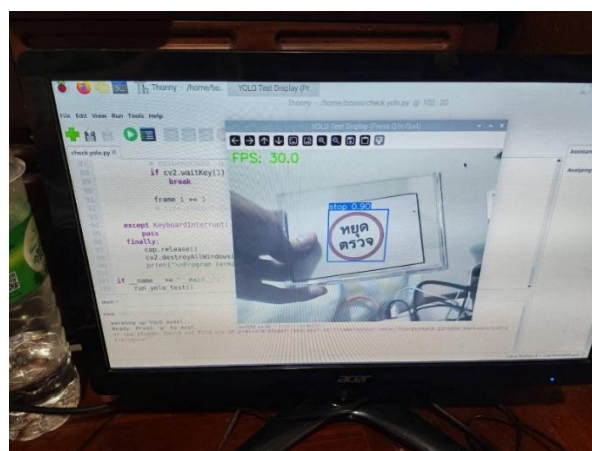
รูปการทดสอบเมื่อกล้องเจอบ่ายตรงไปข้างหน้า ให้รถทำการวิ่งตรงไปข้างหน้า



รูปการทดสอบอัลกอริทึมตรวจจับป้ายเลี้ยวขวา ผลการทดสอบความแม่นยำดีเยี่ยม



รูปการทดสอบอัลกอริทึมตรวจจับป้ายเลี้ยวซ้าย ผลการทดสอบความแม่นยำดีเยี่ยม



รูปการทดสอบอัลกอริทึมตรวจจับป้ายหยุด ผลการทดสอบความแม่นยำดีเยี่ยม

ตารางค่าF1 Score

ป้าย	ครั้งที่1	ครั้งที่2	ครั้งที่3	ครั้งที่4	ครั้งที่5	ครั้งที่6	ครั้งที่7	ครั้งที่8	ครั้งที่9	ครั้งที่10
เดิน	0.89	0.87	0.91	0.92	0.90	0.85	0.94	0.92	0.91	0.91
ซ้าย	0.88	0.89	0.93	0.93	0.87	0.88	0.84	0.92	0.91	0.90
ขวา	0.90	0.90	0.91	0.88	0.89	0.88	0.88	0.93	0.87	0.89
หยุด	0.94	0.93	0.94	0.85	0.90	0.96	0.91	0.89	0.89	0.89

บทที่ 5

บทสรุปและข้อแนะนำ

5.1 บทสรุปโครงการ

โครงการนี้เป็นระบบขับเคลื่อนอัตโนมัติที่ผสมผสานการควบคุมฮาร์ดแวร์มอเตอร์เข้ากับการประมวลผลภาพขั้นสูงด้วยโมเดล YOLO เพื่อแปลผลการตรวจจับวัตถุเป็นคำสั่งเคลื่อนที่ หลักการควบคุมมอเตอร์มีความชัดเจนและเป็นไปตามมาตรฐานการขับเคลื่อนด้วย H-Bridge ถูกควบคุมอย่างแม่นยำผ่านสัญญาณ PWM (Pulse Width Modulation) โดยการปรับค่า Duty Cycle (0-100) บนขาควบคุมความเร็ว ENA/ENB ของแผงควบคุมมอเตอร์ทิศทาง ถูกกำหนดโดยการตั้งค่าแรงดันไฟฟ้า HIGH/LOW บนขาควบคุมทิศทาง IN1-IN4 ซึ่งเป็นการจัดการพื้นฐานของ H-Bridge คำสั่งควบคุมใช้ฟังก์ชันระดับสูง เช่น forward, left, right, stop และแปลงคำสั่งตั้งค่าทิศทาง (+1, -1, 0) ให้มอเตอร์ A และ B

ระบบใช้อัลกอริทึมเพื่อให้การตรวจจับวัตถุและการตอบสนองต่อคำสั่งอย่างรวดเร็วและมีประสิทธิภาพ มีการ Warmup Model YOLO และจะรันภาพว่างหนึ่งครั้งก่อนเริ่มต้นลูปหลัก เพื่อลดความหน่วงจากเฟรมแรก และเตรียมพร้อมสำหรับการประมวลผลจริงในเฟรมถัดไปและเป็นการลดภาระงานของ CPU/GPU และเพิ่มความเร็วในการตอบสนองโดยรวมของระบบ

เทคนิค Command Throttling ถูกใช้เพื่อป้องกันการสั่งงานมอเตอร์ซ้ำถี่เกินไปจนเกิด Flickering หรือการเปลี่ยนทิศทางอย่างรวดเร็ว ระบบใช้ตัวแปร COMMAND_HOLD 0.5 วินาทีในการจำกัดความถี่ของคำสั่งมอเตอร์ที่ส่งออก เมื่อสั่ง เลี้ยว (left/right) รถจะหมุนแบบ Pivot ด้วยความเร็ว SPEED_TURN เป็นระยะเวลาสั้นๆ ที่กำหนดโดย TURN_DUR จากนั้นจะกลับมาสั่ง forward ทันที วิธีนี้ช่วยให้รถเลี้ยวในมุมที่ต้องการได้อย่างมีประสิทธิภาพ มีการควบคุม Special Detection Check แม้ว่าการตัดสินใจหลักจะเลือกกล่องที่มีความเชื่อมั่นสูงสุด แต่มีลอจิกพิเศษที่ซับซ้อนหากตรวจพบป้าย เลี้ยวขวา ระบบจะรัน YOLO ซ้ำด้วยค่าเชื่อมั่นต่ำเพื่อตรวจสอบว่ามีป้ายเลี้ยวซ้ายซ่อนอยู่หรือไม่ แต่หากพบป้ายเลี้ยวซ้าย ระบบจะให้ความสำคัญกับคำสั่งเลี้ยวซ้ายแทน ซึ่งเป็นกลไกการเลือกคำสั่งที่บ่งชี้ถึงลำดับความสำคัญของป้าย

ตอนจบมีการใช้เทคนิค Safe Shutdown เป็นมาตรการป้องกันปัญหาฮาร์ดแวร์ เพื่อให้แน่ใจว่าเมื่อโปรแกรมหยุดทำงาน จะมีการสั่งหยุด PWM, ปิดกล้องและคืนค่าขาทำความสะอาด GPIO อยู่เสมอ

5.2 ข้อเสนอแนะ

ควรมีการกำหนดความเร็วที่เหมาะสมไม่เร็วเกินไปจน YOLO ประมวลผลไม่ทันหรือไม่ช้าเกินไปจนรถไม่เกิดความคล่องตัวทำให้อาจมีข้อผิดพลาดทั้งในส่วนของ YOLO time.sleep ควรตั้งให้เหมาะสมไม่น้อยเกินไป

และมากเกินไปจะทำให้รถเลี้ยวผิดพลาดได้ และถ้ามีการติดเส้นเป็น Map เพื่อใช้เซนเซอร์ตรวจจับเส้น IR จะช่วยให้การวิ่งของรถมีประสิทธิภาพมากขึ้นแต่ข้อเสียก็มีถ้าแสงไม่เพียงพอ IR ตรวจจับไม่ได้รถก็จะไม่ทำงานเหมือนกัน และอีกข้อแหล่งจ่ายไฟ ต้องมีเสถียรภาพและสามารถจ่ายกระแสเพียงพอ เพื่อป้องกันไม่ให้เกิดความเร็วตกหรือบอร์ดกระชากไฟซึ่งอาจจะทำให้บอร์ดได้รับความเสียหายได้ หรือมีการใส่บอร์ด regulator ปรับแรงดันไฟก็จะไม่ต้องใส่แหล่งจ่ายไฟหลายตัวสามารถมีแค่แหล่งจ่ายไฟเดียวในการเลี้ยงบอร์ดควบคุมและโมดูลขับเคลื่อนมอเตอร์ได้เลย และตัวรถเนื่องจากเป็นรถที่เป็น demo ชุด kit ทำให้มอเตอร์ไม่มี Encoder ตัวรถเดินตามป้ายจราจรจะมีการเอียงเล็กน้อยดังนั้นตัวรถจึงเหมาะกับการเดินตามป้ายในระยะทางสั้น ๆ หรือในสภาพแวดล้อมที่มีการปรับแก้บ่อยครั้ง

แหล่งอ้างอิงของโครงการ

- 1 <https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf>
- 2 <https://docs.opencv.org/4.x/d1/dfb/intro.html>
- 3 <https://www.electronicwings.com/raspberry-pi/raspberry-pi-pwm-generation-using-python-and-c>
- 4 <https://www.pololu.com/category/11/motor-drivers-and-controllers>
- 5 <https://www.st.com/resource/en/datasheet/l298.pdf>
- 6 <https://www.ti.com/lit/ds/symlink/l293.pdf>
- 7 <https://www.raspberrypi.com/documentation/accessories/camera.html>
- 8 <https://www.opencv.org/>
- 9 <https://github.com/tzutalin/labelImg>
- 10 <https://cocodataset.org/#format-data>
- 11 <https://pytorch.org/docs/stable/index.html>
- 12 <https://learn.sparkfun.com/tutorials/how-to-run-a-raspberry-pi-program-on-startup/all>
- 13 <https://www.thedigitalpictureframe.com/ultimate-guide-systemd-autostart-scripts-raspberry-pi/>
- 14 https://docs.opencv.org/4.x/d8/dfe/classcv_1_1VideoCapture.html
- 15 <https://blog.aarav.com.np/creating-and-understanding-a-systemd-service-file-on-raspberry-pi>
- 16 <https://www.collvy.com/blog/controlling-dc-motor-with-raspberry-pi>
- 17 <https://www.embedded.com/architecture-of-safety-critical-systems/>